

Set theory

 This chapter overlaps with sections 2.1, 2.2, 2.3, 9.1 and 9.3 of Rosen.

1.1 Sets

1.1.1 Definitions

Definition 1.1 (Set and elements)

A set is a collection of objects which are called elements of the set. A set is fully determined by *which* elements compose it.

Notation

- If a is an element of the set A , we write $a \in A$.
- If an object a is not an element of the set A , we write $a \notin A$.

How to describe a set?

- By using a roster, listing all elements of the set, either explicitly like:

$$S_1 = \{1, 2, 3, 4, 5, 6\}$$

or implicitly, using an ellipsis:

$$S_1 = \{1, 2, \dots, 6\}$$

- By using a defining predicate:

$$S = \{y \in S_0 \mid P(y)\}$$

where $P(y)$ is a condition on the elements y of another set S_0 . Then S is the set of all elements y of S_0 such that $P(y)$ is true. For instance:

$$S_2 = \{y \in S_1 \mid y < 3\}$$

- Through a transformation f of the elements y of another set S_0 :

$$S = \{f(y) \mid y \in S_0\}$$

For instance:

$$S_3 = \{n^3 \mid n \in S_1\}$$

The notation “ $\mid$ ” is read as “such that”. It can also be written with a colon “ $:$ ”.

You may also combine the predicate and transformation methods, but it’s very rarely necessary.

Note that there is often more than one way to define the same set. However, there is often a better choice, at least in terms of clarity. For instance, how would you write S_3 above using a predicate on the set of natural numbers?

❓ Question

From [Definition 1.1](#), especially its second part, can you guess the condition for two sets to be equal? For instance, are $A = \{1, 2\}$, $B = \{2, 1\}$ and $C = \{1, 1, 2\}$ equal?

Definition 1.2 (Sets equality)

Two sets are equal if and only if they have the same elements.

This definition of sets equality actually also indirectly defines what a set is.

Definition 1.3 (Empty set $\emptyset$)

The **empty set** $\emptyset$ is the set with no elements: $\emptyset = \{\}$.

Definition 1.4 (Universal set U)

The **universal set** U is the set containing all objects under consideration.

This concept can be useful to keep notation consistent. For instance, to formally write sets equality: two sets A and B are equal if and only if

$$\forall x \in U, x \in A \Leftrightarrow x \in B,$$

instead of

$$\forall x \in ?, x \in A \Leftrightarrow x \in B.$$

Most of the time, and in the following of this course, $x \in U$ can be omitted for convenience, though.

Definition 1.5 (Set finiteness and cardinality)

Let S be a set. If there are exactly $n \in \mathbb{N}$ distinct elements in S , we say that S is a **finite set**, and that $|S| = n$ is the **cardinality** of S .

1.1.2 Examples: some already-known sets

Definition 1.6 (Natural numbers)

The set of natural numbers $\mathbb{N}$ is defined by the following conditions:

- (i) $0 \in \mathbb{N}$.
- (ii) If $n \in \mathbb{N}$, then the successor of n (i.e., the number $n + 1$) belongs to $\mathbb{N}$.
- (iii) Every $n \in \mathbb{N}$ except 0 is the successor of some number in $\mathbb{N}$.
- (iv) Every non-empty subset of $\mathbb{N}$ has a minimum element.

Notes:

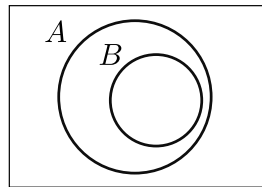
- Some define the natural numbers starting from one, thus excluding zero. To avoid any ambiguity, one can respectively use the terms “non-negative” or “positive” integers to distinguish between the definition of natural numbers including or excluding zero.
- We can informally “define” the following sets of numbers:
 - Integer numbers: $\mathbb{Z} = \{0, \pm 1, \pm 2, \dots\}$
 - Rational numbers: $\mathbb{Q} = \left\{ \frac{p}{q} \mid p, q \in \mathbb{Z}, q \neq 0 \right\}$
- These sets are discrete yet infinite: their cardinality is undefined (or infinite).

1.1.3 Subsets

Definition 1.7 (Subset)

The set A is a **subset** of the set B ($A \subseteq B$) if and only if every element of A is also an element of B . The set A is a **proper subset** of B ($A \subset B$) if A is a subset of B , and B contains at least an element not in A .

We can represent $A \subseteq B$, or $A \subset B$ here to be precise, with a **Venn diagram**:



Properties:

- The empty set $\emptyset$ is a subset of every set A : $\emptyset \subseteq A$.
- Every set A satisfies $A \subseteq A \subseteq U$.

Important

The latter implies that $A = B \Leftrightarrow (A \subseteq B) \wedge (B \subseteq A)$. This is the basis for how we usually prove the equality of two sets:

- Consider $x \in A$, prove it is also in B .
- Consider $x \in B$, prove it is also in A .

Definition 1.8 (Power set)

The **power set** of the set A , denoted as $\mathcal{P}(A)$, is the set of all subsets of A :

$$\mathcal{P}(A) = \{B \mid B \subseteq A\}$$

1.2 Set operations

Given two sets A and B we can define a number of operations.

Definition 1.9 (Sets union)

The union of two sets A and B is the set of all elements that are in A , in B , or in both:

$$A \cup B = \{x \mid (x \in A) \vee (x \in B)\}$$

Definition 1.10 (Sets intersection)

The intersection of two sets A and B is the set of all elements that are in both A and B :

$$A \cap B = \{x \mid (x \in A) \wedge (x \in B)\}$$

Notation

These two operations can be chained an arbitrary number of times.

For example we can perform the union of n sets $A_1, A_2, \dots, A_n$ and use the following notation:

$$A_1 \cup A_2 \cup \dots \cup A_n = \bigcup_{i=1}^n A_i,$$

or perform the intersection of the same sets:

$$A_1 \cap A_2 \cap \dots \cap A_n = \bigcap_{i=1}^n A_i.$$

These two notations are the respective equivalents of the sum $\sum$ and product $\prod$ notations that you already knew.

Definition 1.11 (Set complement)

The complement of a set A is the set of all elements that are not in A :

$$\overline{A} = \{x \mid x \notin A\} = U \setminus A$$

Definition 1.12 (Sets difference)

The difference of two sets A and B is the set of all elements that are in A but not in B :

$$A \setminus B = \{x \mid (x \in A) \wedge (x \notin B)\}$$

Definition 1.13 (Sets symmetric difference)

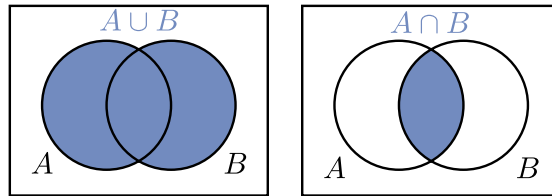
The symmetric difference of two sets A and B is the set of all elements that are in A or in B but not in both:

$$A \triangle B = \{x \mid (x \in A \cup B) \wedge (x \notin A \cap B)\}$$

A set can therefore be defined as the result of operations involving other sets. For instance,

$$\mathbb{Z} = \mathbb{N} \cup \{-n \mid n \in \mathbb{N}\}, \quad \mathbb{Z}^+ = \mathbb{N} \setminus \{0\}$$

You can represent these operations with a Venn diagram too. For instance:



? Question

How would you represent the other operations? Then, using these representations, how would you write the difference operations as combinations of the union, intersection and complement operations?

Properties 1.14

- **Distributive laws:**
 - $A \cup (B \cap C) = (A \cup B) \cap (A \cup C)$
 - $A \cap (B \cup C) = (A \cap B) \cup (A \cap C)$
- **De Morgan's laws:**
 - $\overline{A \cup B} = \overline{A} \cap \overline{B}$
 - $\overline{A \cap B} = \overline{A} \cup \overline{B}$
- $A \triangle B = (A \setminus B) \cup (B \setminus A)$

You can check these laws hold with Venn diagrams too.

Definition 1.15

Two sets A and B are **disjoint** if $A \cap B = \emptyset$.

1.3 Applications

1.3.1 Data types as sets

The very concept of a datatype in computer science is built upon the definition of a set. For instance, a boolean is an element of the set $\{0, 1\}$, or, equivalently, $\{\text{False}, \text{True}\}$. A datatype is more than a set though, as it also defines the operations which are allowed on the elements of the set.

? Question

Note that all data types correspond to a set which is *necessarily* finite. Can you guess why?

1.3.2 Sets as a data type

Most programming languages define a set datatype. 🐍 **Python is no exception**: you can define a set with a syntax that is very similar to the notation we have seen until now:

`S = {'a', 1, {'b'}}`

Try at home

Find the equivalents of the set operations we saw above in the [Python documentation](#), and try them out! Also check that sets in Python have the properties listed above, in particular that the ordering of elements does not matter.

1.4 Relations

Definition 1.16 (Tuple)

An n -tuple is an ordered collection of n objects, of the form $(a_1, a_2, \dots, a_n)$. A 2-tuple is often called an ordered pair.

This means that, contrary to sets, for tuples the ordering of elements matters. Thus, while $\{a_1, a_2\} = \{a_2, a_1\}$, for tuples $(a_1, a_2) \neq (a_2, a_1)$.

Definition 1.17 (Cartesian product)

Given two sets A and B , the **Cartesian product** $A \times B$ is the set of all ordered pairs (a, b) where $a \in A$ and $b \in B$. Formally, that is:

$$A \times B = \{(a, b) \mid (a \in A) \wedge (b \in B)\}$$

Remark

A Cartesian product gives you all the possible combinations involving one element from each set. If $A = B$, it gives all the possible combinations of two elements in this set. You already saw Cartesian products in calculus, when you wrote, for instance, $\forall (x, y) \in \mathbb{R}^2$: these are all the possible pairs of real numbers. Then, you used the notation $\mathbb{R}^2 = \mathbb{R} \times \mathbb{R}$.

Definition 1.18 (Binary relations)

A **binary relation** R from the set A to the set B is a set of ordered pairs from the two sets. Said differently, R is a subset of $A \times B$, so: $R \subseteq A \times B$.

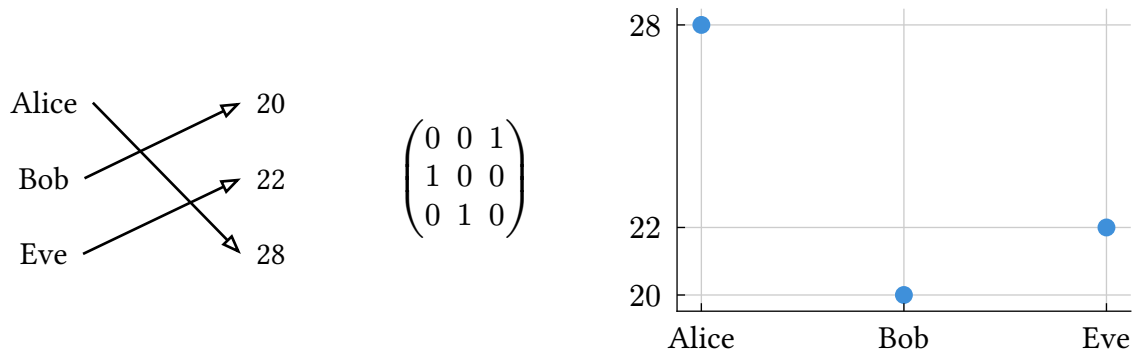
The set A is then called the **domain** of R while B is called its **codomain**.

If $a \in A$ is related to $b \in B$ by R , we therefore have $(a, b) \in R$, which we can also write $a R b$. If they are not related, we have $(a, b) \notin R$, which we can also write $a \not R b$.

How can we represent binary relations graphically? What we want is to show each set on a separate “side”, and link related elements together. Here we’ll see three options, which are the directed graph, Cartesian and adjacency matrix representations.

Example. Let’s consider that we know three persons, called Alice, Bob and Eve, who are respectively 28, 20 and 22 years-old. We want to represent this information as a binary

relation. We can then consider the set $A = \{\text{Alice}, \text{Bob}, \text{Eve}\}$ of people's names and the set $B = \{20, 22, 28\}$ of some ages. We can thus define the relation R as $a R b$ if person $a \in A$ has the age $b \in B$, and represent it as:



❓ Question

- When is the Cartesian representation preferable? Hint: imagine if we took $B = \mathbb{N}$.
- Imagine other relations in which the individual relations are not one-to-one, but one-to-many, many-to-one or many-to-many.

Definition 1.19 (Inverse relations)

Let R be a relation from the set A to the set B . The inverse relation of R is the relation that assigns to each element $b \in B$ an element $a \in A$. We denote the inverse by R^{-1} , so that

$$\forall (a, b) \in A \times B, a R b \Leftrightarrow b R^{-1} a$$

💬 Remark

All relations have an inverse.

Definition 1.20 (Composition of relations)

Let R be a relation from the set A to the set B , and let S be a relation from the set B to the set C . The composition of the relation S and R is a relation from A to C denoted as $S \circ R$. In particular, $S \circ R$ is a subset of the Cartesian product $A \times C$ such that, given any $a \in A$ and $c \in C$, $a(S \circ R)c$ if and only if there exists some $b \in B$ satisfying $a R b$ and $b S c$.

Proposition 1.21

Let A_R be the adjacency matrix of the relation R from the set A to the set B , and let A_S be the adjacency matrix of the relation S from the set B to the set C . Then, the adjacency matrix $A_{S \circ R}$ of the composition of the relations $S \circ R$ from A to C is given by:

$$A_{S \circ R} = A_R \odot A_S$$

where the product $\odot$ is the Boolean product of matrices.

Using Boolean operations (instead of regular ones) guarantees that $A_{S \odot R}$ is an adjacency matrix associated to a binary relation.

1.5 Defining functions as relations

Definition 1.22 (Functions)

A binary relation f with a domain X and codomain Y is said to be a **function** if it assigns exactly one element of Y to each element of X .

Notation

A function $f \subset X \times Y$ is usually defined with the notation $f : X \rightarrow Y$, to signify that it maps each element of X to a unique element of Y .

Also, we write $f(x) = y$ if y is the unique element of Y assigned by f to the element x of X .

Definition 1.23 (Function images and range)

Let's consider $f : X \rightarrow Y$ and $(x, y) \in X \times Y$ such that $f(x) = y$. We then say that y is the **image** of x under the function, and that x is the **preimage** of y . We also define the **range** of the function as the set of images of every element in the domain X , and often write this range as $f(X)$. The range is therefore a subset of the codomain: $f(X) \subseteq Y$.

Definition 1.24

Given a function $f : X \rightarrow Y$, we say that

- f is **injective** or **one-to-one** if $x_1 \neq x_2$ implies $f(x_1) \neq f(x_2)$.
- f is **surjective** if for every $y \in Y$, there exists at least an element $x \in X$ such that $y = f(x)$.
- f is **bijective** if it is injective and surjective.

Definition 1.25 (Inverse function)

If $f : X \rightarrow Y$ is bijective, we can define its **inverse function** $f^{-1} : Y \rightarrow X$ by the well-defined rule:

$$f^{-1}(y) = x \iff y = f(x)$$

Question

While all relations have an inverse, a function, which is a particular kind of relation, only has an inverse if it is bijective. How come?

Definition 1.26 (Composition of functions)

Given two functions $f : X \rightarrow Y$, $g : Y \rightarrow Z$, we can define a new function $g \circ f : X \rightarrow Z$ by the following rule:

$$(g \circ f)(x) = g(f(x))$$

The function $g \circ f$ is the **composition** of f and g .

Relations

☞ This chapter overlaps with sections 9.1, 9.5, 9.6, 5.1 and 5.2 of Rosen.

2.1 Binary relations on a set

While we previously introduced relations R from a set A to a set B , here we'll be particularly interested in the case where $A = B$, that is relations on a single set.

Definition 2.1 (Binary relation on a set)

A **binary relation** R on the set A is a subset of $A \times A$, so: $R \subseteq A \times A$.

In the following, unless otherwise specified, “a relation R ” refers to a relation on a single set A .

Definition 2.2 (Reflexive relations)

A relation R is reflexive if all elements of A are related to themselves:

$$\forall a \in A, a R a$$

A relation R is irreflexive if all elements of A are not related to themselves:

$$\forall a \in A, a \not R a$$

Definition 2.3 (Symmetric relations)

A relation R is symmetric if $R = R^{-1}$, i.e., if

$$\forall (a, b) \in A^2, a R b \Rightarrow b R a$$

A relation R is antisymmetric if

$$\forall (a, b) \in A^2 \text{ such that } a \neq b, a R b \Rightarrow b \not R a$$

Definition 2.4 (Transitive relations)

A relation R is transitive if

$$\forall (a, b, c) \in A^3, (a R b) \wedge (b R c) \Rightarrow a R c$$

Proposition 2.5

A relation R is transitive if and only if $R^n \subseteq R$ for all $n \in \mathbb{N}$. The n -th power R^n of the relation R is recursively defined as follows:

$$R^1 = R, \quad R^n = R \circ R^{n-1}$$

Corollary 2.5.1

A relation R is transitive if and only if $R^2 \subseteq R$. In other words, R is transitive if and only if for each nonzero entry $(A_{R^2})_{i,j} = 1$ of the adjacency matrix of R^2 , the corresponding entry of the adjacency matrix of R is also nonzero $(A_R)_{i,j} = 1$.

2.2 Equivalence relations

Definition 2.6 (Equivalence relations)

A relation R on a set A is an equivalence relation if it is reflexive, symmetric and transitive.

Notation

If R is an equivalence relation, $a R b$ is usually denoted as $a \equiv_R b$ or $a \sim_R b$.

The R subscript can be dropped when there is no ambiguity on which equivalence relation we're dealing with.

Question

What is the simplest equivalence relation that you know?

Example. Is the following relation on the set P of all people an equivalence relation?

$$R_P = \{(a, b) \in P^2 \mid a \text{ is the brother of } b\}$$

If not, what's the closest relation you can think of that's actually an equivalence?

Definition 2.7 (Equivalence classes)

Let R be an equivalence relation on a set A . The set of all the elements of A related to a certain element $a \in A$ is called the **equivalence class** of a with respect to R , and it is denoted as $[a]_R$, or simply as $[a]$. Therefore,

$$[a]_R = \{b \in A \mid a R b\}$$

Any element $b \in [a]_R$ is called a **representative** of the equivalence class of a .

Theorem 2.8

Let R be an equivalence relation on A . Then,

- (i) $[a]_R$ is non-empty for all $a \in A$.
- (ii) For any two elements $a, b \in A$, either $[a]_R = [b]_R$ (and $a R b$), or $[a]_R \cap [b]_R = \emptyset$.
- (iii) The equivalence classes determine the relation uniquely.

Equivalence classes are useful because they allow us to partition a set. But first, what does that even mean?

Definition 2.9 (Set partition)

A partition of a set A is a set of disjoint nonempty subsets of A whose union form A .

In other words, a set of subsets $\{A_i\}_{i \in I}$ such that:

- (i) $\forall i \in I, A_i \neq \emptyset$,
- (ii) $\forall (i, j) \in I^2, A_i \cap A_j = \emptyset$ if $i \neq j$,
- (iii) $\bigcup_{i \in I} A_i = A$,

and where I is a (potentially infinite!) set of indices.

Theorem 2.10

Let R be an equivalence relation on A . Then the set of all equivalence classes of R form a partition of A . Conversely, given a partition $\{A_1, A_2, \dots\}$ of A , there exists an equivalence relation R such that its equivalence classes are the sets A_i .

❓ Question

If there exists $a, b \in A$ such that $a \equiv b$, then $[a]_R = [b]_R$, which means $[a]_R \cap [b]_R \neq \emptyset$. Since equivalence classes partition a set, doesn't that contradict the fact that the subsets composing a partition should be disjoint?

Example. From the equivalence relation you defined in the example above. translate in plain words what its associated equivalence classes represent. Then also explain in plain words why they do form a partition of the set of all people, that is how they fulfill each condition of Definition 2.9.

Definition 2.11 (Quotient set)

Let R be an equivalence relation on A . The set of all the equivalence classes of R is called the quotient set of A by R , and it is denoted by A/R :

$$A/R = \{[a]_R \mid a \in A\}$$

2.3 Order relations

Definition 2.12 (Partial order relation)

A relation on a set A is called a **partial order** if it is reflexive, antisymmetric, and transitive.

Notation

Order relations are usually denoted by the symbol $\preceq$.

We write $a \prec b$ to denote that $a \preceq b$ and $a \neq b$, and then say that “ a precedes b ”.

Definition 2.13 (Posets)

A set A equipped with an order relation $\preceq$ is called a **partially ordered set**, or poset, and is denoted by $(A, \preceq)$.

Example. Are the following posets?

- (i) $(\mathbb{Z}, \geq)$
- (ii) $(\mathbb{N}, |)$, where $|$ is the “divides” relation: $a \mid b \Leftrightarrow \exists q \in \mathbb{N}$ such that $b = q \cdot a$.
- (iii) $(P, \{(a, b) \in P \mid a \text{ is older than } b\})$, where P is the set of all people.

Since a partial order is a relation, by definition it does not impose any constraint between unrelated elements. It is therefore useful to distinguish between elements which can be compared using an ordering, and those which cannot.

Definition 2.14 (Comparability)

Let $(A, \preceq)$ be a partially ordered set. Two elements $a, b \in A$ are said to be **comparable** if either $a \preceq b$ or $b \preceq a$. This is basically just another word for “related”, but which makes more intuitive sense in the context of orderings. If neither of these conditions holds, such elements are said to be **incomparable**.

The adjective “partial” in “partial ordering” thus refers to the fact that the ordering does not necessarily order all elements of its associated set. But if it is the case, the poset has some interesting properties, and so we have a special term to refer to them.

Definition 2.15 (Total order)

A partially ordered set $(A, \preceq)$ is said to be **totally ordered** when any two elements of A are comparable.

2.3.1 Representing posets: Hasse diagrams

We saw in Section 1.4 that relations can be represented by a directed graph, by drawing an arrow from $a \in A$ to $b \in B$ if $a R b$. A poset can therefore also be represented as a directed graph, but with several simplifications due to its properties.

- (i) It is a relation on a single set ($A = B$), so elements only need to be represented once.
- (ii) It is antisymmetric, so we can fix a convention such as “if $a \prec b$, then a will be represented below b ”. This implies that all arrows point in the same direction, upwards, so they can be omitted.
- (iii) It is reflexive, so the loops corresponding to $a \preceq a$ do not need to be represented.
- (iv) It is transitive, so the edge corresponding to $a \prec c$ does not need to be represented if there exists b such that $a \prec b$ and $b \prec c$.

All these simplifications lead to a representation for posets known as a **Hasse diagram**.

2.3.2 Extremal elements

Definition 2.16 (Extremal elements)

Let $(A, \preceq)$ be a partially ordered set.

- $M \in A$ is a maximal element if it does not precede any other element:

$$\forall a \in A, M \preceq a \Rightarrow a = M$$

- $m \in A$ is a minimal element if no other element precedes it:

$$\forall a \in A, a \preceq m \Rightarrow a = m$$

In other words, in the Hasse diagram associated to $(A, \preceq)$, there is no element above M , and no element below m .

Definition 2.17 (The maximum and minimum)

Let $(A, \preceq)$ be a partially ordered set.

- $M^* \in A$ is **the maximum (or greatest element)** of A if all elements precede or are equal to it:

$$\forall a \in A, a \preceq M^*$$

- $m^* \in A$ is **the minimum (or least element)** if it precedes or is equal to all elements:

$$\forall a \in A, m^* \preceq a$$

In other words, in the Hasse diagram associated to $(A, \preceq)$, M^* is above all the elements of A , and m^* is below all elements of A . The maximum and minimum of $(A, \preceq)$ are denoted by $\max(A)$ and $\min(A)$, respectively.

Remark

The maximal, minimal, greatest and least elements of $(A, \preceq)$ might not exist.

Example. Do the following posets have extremal elements and a maximum and minimum?

- (i) $(\{2, 5, 3, 7\}, |)$,
- (ii) $(\{2, 3, 4, 9\}, |)$,
- (iii) $(\{2, 4, 6, 8\}, |)$,

(“ $|$ ” is the “divides” relation from the example above).

Theorem 2.18

The maximum M^* of a partially ordered set $(A, \preceq)$, if it exists, is unique. In addition, the maximum of $(A, \preceq)$ is also a maximal element of it.

Remark

To be the maximum, a maximal element needs to be comparable to all other elements!

Definition 2.19 (Bounds)

Let $(A, \preceq)$ be a partially ordered set, and $B \subset A$.

- $u \in A$ is an upper bound of B if $b \preceq u$ for all $b \in B$.
 - The set of the upper bounds of B is denoted by $\text{major}(B)$.
 - The supremum of B , $\sup(B)$, is the least upper element of B :

$$\sup(B) = \min(\text{major}(B))$$

- $l \in A$ is a lower bound of B if $l \preceq b$ for all $b \in B$.
 - The set of all the lower bounds of B is denoted by $\text{minor}(B)$.
 - The infimum of B , $\inf(B)$, is the greatest lower element of B :

$$\inf(B) = \max(\text{minor}(B))$$

Remark: It may happen that $\text{major}(B) = \emptyset$, $\text{minor}(B) = \emptyset$ and/or $\sup(B)$ and $\inf(B)$ do not exist.

2.3.3 Well-ordering and induction

Definition 2.20

$(A, \preceq)$ is a well-ordered set if $(A, \preceq)$ is a total order and all nonempty subsets of A have a minimum.

Remark

- The set of natural numbers with the usual order $(\mathbb{N}, \leq)$ is a well-ordered set.
- The totally-ordered set $(\mathbb{Z}, \leq)$ is not a well-ordered set; but as $\mathbb{Z}$ is isomorphic to $\mathbb{N}$, we can choose another order $\preceq$ such that $(\mathbb{Z}, \preceq)$ is a well-ordered set.

The well-ordering of natural numbers then explains the validity of proofs by induction!

Theorem 2.21 (Induction principle: weak version)

Let P be some predicate on the positive integers. Under the following conditions:

Basis step We can show that $P(n_0)$ is true for a fixed $n_0 \in \mathbb{N}$.

Inductive step If we assume that $P(k)$ is true for some unspecified $k \geq n_0$, then we can show that $P(k+1)$ is true.

$P(n)$ is true for every $n \geq n_0$.

Theorem 2.22 (Induction principle: strong version)

Let P be some predicate on the positive integers. Under the following conditions:

Basis step We can show that $P(n_0)$ is true for a fixed $n_0 \in \mathbb{N}$.

Inductive step If we assume that for some unspecified $k \geq n_0$, $P(j)$ is true for any $n_0 \leq j \leq k$, then we can show that $P(k+1)$ is true.

$P(n)$ is true for every $n \geq n_0$.

Remark

n_0 is very often 0 or 1, but not always!

Example. Find n_0 such that $2^n \geq n + 5$ for all $n \geq n_0$.

Theorem 2.23 (Strong induction principle for well-ordered sets)

Let $(A, \preceq)$ be a well-ordered set. Under the following condition:

Inductive step If we assume that for any b in A , for all its predecessors $a \prec b$, $P(a)$ is true, then we can show that $P(b)$ is true.

$P(a)$ is true for every $a \in A$.

Remark

There is no basis step here because since A is well-ordered, we know that its least element a_0 has no predecessor ($\{a \in A \mid a \prec a_0\} = \emptyset$), so the inductive step suffices to prove that $P(a_0)$ is true.

2.4 Summary: types of relations

Relation	Reflexive	Symmetric	Antisymmetric	Transitive	Additional Properties
Equivalence	✓	✓	✗	✓	
Order	✓	✗	✓	✓	
Total order	✓	✗	✓	✓	Every pair is comparable
Well-ordered set	✓	✗	✓	✓	Every non-empty subset has a minimum

You will see different kinds of relations again in a more or less close future.

- In relational databases, each table defines a relation between n sets of attributes, where the values for each set are stored in a column (see Rosen 9.2 for further reading).

- A particular case of equivalence relation and its associated classes will be studied in Section 3.2.
- Order relations constitute the formalism on which all sorting algorithms are built, whether they concern numbers or strings of text.

Elementary number theory

 This chapter overlaps with sections 4.1, 4.3, 4.5 and 4.6 of Rosen.

3.1 Integer divisibility

The set of integers $\mathbb{Z}$ is closed with respect to the operations of sum, subtraction, and product. In other words, for every $a, b \in \mathbb{Z}$, $a \pm b \in \mathbb{Z}$ and $a \cdot b \in \mathbb{Z}$. They also satisfy:

- 0 is the identity with respect to the sum: $a + 0 = a$ for every $a \in \mathbb{Z}$.
- 1 is the identity with respect to the product: $a \cdot 1 = a$ for every $a \in \mathbb{Z}$.
- For every $a \in \mathbb{Z}$, there exists a unique inverse element $-a \in \mathbb{Z}$ such that $a + (-a) = 0$.

However, the result of dividing two integers might not be an integer.

Definition 3.1 (Divisibility)

Given two integers $a \neq 0$ and b , we say that a **divides** b if there exists an integer $q \in \mathbb{Z}$ such that $b = a \cdot q$. If a divides b , we say that a is a **factor** or **divisor** of b and that b is a **multiple** of a . We denote $a \mid b$ when a divides b , and we write $a \nmid b$ when a does not divide b .

Remark

- Every non-zero integer divides 0:

$$\forall a \in \mathbb{Z} \setminus \{0\}, 0 = a \cdot 0 \quad (q = 0).$$

- Every nonzero integer divides itself:

$$\forall a \in \mathbb{Z} \setminus \{0\}, a = a \cdot 1 \quad (q = 1).$$

- 1 divides any integer:

$$\forall a \in \mathbb{Z}, a = 1 \cdot a \quad (q = a).$$

Theorem 3.2

Let a, b, c be integers. Then:

- If $a \mid b$ and $a \mid c$, then $a \mid (b + c)$.
- If $a \mid b$, then $a \mid (b \cdot c)$ for every $c \in \mathbb{Z}$.
- If $a \mid b$ and $b \mid c$, then $a \mid c$.
- If $c \neq 0$, then $a \mid b$ if and only if $(c \cdot a) \mid (c \cdot b)$.
- If $a \mid b$ and $b \neq 0$, then $|a| \leq |b|$.
- If $a \mid b$ and $b \mid a$, then $a = \pm b$.

Theorem 3.3

If $a \mid b_i$ for $i = 1, \dots, N$, then $a \mid \sum_{i=1}^N u_i \cdot b_i$ for all $u_i \in \mathbb{Z}$. In other words, any integer divides any linear combination with integer coefficients of its multiples.

Theorem 3.4 (The Division Algorithm)

Let a and $b \neq 0$ be two integers. Then there exists a unique pair of integers q and r such that

$$a = q \cdot b + r \quad \text{with } 0 \leq r < |b|$$

Definition 3.5

In the equality given in [Theorem 3.4](#):

- the numbers a and b are called dividend and divisor, respectively,
- the number r is called the **remainder** and we write

$$r = a \bmod b,$$

- the number q is called the **quotient** and we write

$$q = a \operatorname{div} b = \begin{cases} \lfloor a/b \rfloor & \text{if } b > 0 \\ \lceil a/b \rceil & \text{if } b < 0 \end{cases}$$

Remark

$\bmod$ is called the modulo or remainder operation, and div the floor division or quotient operation. They both have associated symbols in most programming languages (respectively `%` and `//` in  Python, for instance).

3.2 Modular Arithmetic

Modular arithmetic allows us to perform algebraic operations using, instead of a given set of numbers, their respective remainders with respect to some fixed positive number called the modulus. This might sound convoluted but is actually fairly common. For example, to answer the question “What day of the week will we be 10 days from now?”, you use modular arithmetic. We’ve already seen the operation notation $r = a \bmod b$, but to perform module arithmetic we need to introduce a related notation, which involves a relation instead.

Definition 3.6 (Congruence)

Let a, b be integers, and let m be a natural number. Then a, b are congruent modulo m if $m \mid (a - b)$. This relation is denoted as $a \equiv b \pmod{m}$. It is called a congruence with modulus m .

Proposition 3.7

- (i) $a \equiv b \pmod{m}$ if and only if $a \bmod m = b \bmod m$.
- (ii) $a \equiv b \pmod{m}$ if and only if $a = b + k \cdot m$ for some $k \in \mathbb{Z}$.

The congruence modulo m between two integers thus indicates that they have the same remainder when they are divided by the modulus m .

? Question

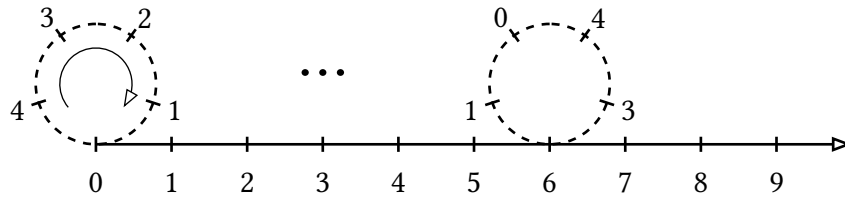
- Can you now translate “What day of the week will we be 10 days from now?” in modular arithmetic terms? Start with wondering: how can I represent days of the week numerically? Then, what is the modulus, here?
- Can you find other examples of real-life problems involving congruences?

Theorem 3.8

For each positive integer m , the binary relation $\equiv (\text{mod } m)$ is an equivalence relation, as defined in Section 2.2.

3.2.1 The Quotient Set $\mathbb{Z}_m$

A good way to visualize such problems of modular arithmetic is to imagine that the straight lines of integers gets rolled into a single circle with m ticks:



This is equivalent to defining the quotient set we'll introduce below!

Definition 3.9 (Congruence classes)

We define the congruence classes modulo m as

$$[a]_m = \{b \in \mathbb{Z} \mid a \equiv b(\text{mod } m)\} = \{a + mk \mid k \in \mathbb{Z}\}$$

Property 3.10

It follows from [Theorem 3.8](#) that congruence classes are equivalence classes, and thus from [Theorem 2.10](#) that they form a partition of $\mathbb{Z}$.

Also, there are m distinct equivalence classes corresponding to the m possible remainders obtained by dividing an integer by m .

Theorem 3.11

The quotient set $\mathbb{Z}_m = \mathbb{Z} / \equiv (\text{mod } m)$ is given by

$$\mathbb{Z}_m = \{[a]_m \mid 0 \leq a \leq m - 1\}$$

Remark: Usually, the notation for $\mathbb{Z}_m$ is a bit sloppy:

$$\mathbb{Z}_m = \{0, 1, 2, \dots, m - 1\}$$

3.2.2 Modular Arithmetic

Theorem 3.12

Let m be a positive integer. If $a_1 \equiv b_1 \pmod{m}$ and $a_2 \equiv b_2 \pmod{m}$, then:

- $a_1 \pm a_2 \equiv b_1 \pm b_2 \pmod{m}$.
- $a_1 \cdot a_2 \equiv b_1 \cdot b_2 \pmod{m}$.

Corollary 3.12.1

Let m, k be positive integers, and let $a, b \in \mathbb{Z}$. If $a \equiv b \pmod{m}$, then $a^k \equiv b^k \pmod{m}$.

3.3 Greatest common divisors and least common multiples

3.3.1 Definitions

Definition 3.13 (Greatest common divisor)

Let a, b be integers which are not both zero. The largest positive integer d that divides both a and b is called the **greatest common divisor** of a and b . It is denoted by $\gcd(a, b)$.

Remark

- The case $a = b = 0$ is excluded because any integer divides 0.
- $\gcd(0, a) = |a|$ for all nonzero integers a .
- $\gcd(a, b) = \gcd(|a|, |b|)$

Definition 3.14 (Relative primes)

Two integers a and b are **relatively prime** if $\gcd(a, b) = 1$.

The integers $a_1, a_2, \dots, a_n$ are pairwise relatively prime if $\gcd(a_i, a_j) = 1$ for any $1 \leq i < j \leq n$.

Definition 3.15 (Least common multiple)

The **least common multiple** of two natural numbers a, b is the smallest natural number m that is divisible by both a and b , that is $a \mid m$ and $b \mid m$. It is denoted by $\text{lcm}(a, b)$.

3.3.2 How to compute them

The gcd can be obtained systematically using the following lemma.

Lemma 3.16

Given the integers $a, b \neq 0, q$ and r , such that $a = q \cdot b + r$ with $0 \leq r < |b|$, then $\gcd(a, b) = \gcd(b, r)$.

But how? Let's write $r_0 = |a|$, $r_1 = |b|$, $r_2 = r$ and $q_1 = q$ and apply the lemma successively:

$$\begin{aligned} r_0 &= q_1 r_1 + r_2 & 0 \leq r_2 < r_1, \\ r_1 &= q_2 r_2 + r_3 & 0 \leq r_3 < r_2, \\ &\vdots \\ r_{n-2} &= q_{n-1} r_{n-1} + r_n & 0 \leq r_n < r_{n-1}, \\ r_{n-1} &= q_n r_n. \end{aligned}$$

Since we have by [Theorem 3.4](#) that $|a| = r_0 > r_1 > \dots > r_n \geq 0$, this sequence of remainders cannot contain more than $|a|$ terms, so it has a finite size n . This means that there exists an n such that $r_n \mid r_{n-1}$. It then follows from [Lemma 3.16](#) that

$$\gcd(a, b) = \gcd(|a|, |b|) = \gcd(r_0, r_1) = \gcd(r_1, r_2) = \dots = \gcd(r_{n-1}, r_n) = r_n$$

Example. Apply recursively Euclid's lemma to compute $\gcd(662, 414)$.

The lcm is then straightforward to compute since we can use the following theorem.

Theorem 3.17

If a, b are two natural numbers, then

$$\gcd(a, b) \cdot \text{lcm}(a, b) = a \cdot b$$

3.3.3 How to use them in modular arithmetic

Theorem 3.18 (Bézout's Identity)

If a and b are two integers not simultaneously zero, then there exist integers x, y such that

$$\gcd(a, b) = a \cdot x + b \cdot y$$

Proof: We write the steps of Euclid's algorithm, and "unroll them":

$$\begin{aligned} a &= q_1 \cdot b + r_2 \Rightarrow r_2 = a - q_1 \cdot b \\ b &= q_2 \cdot r_2 + r_3 \Rightarrow r_3 = b - q_2 \cdot r_2 \\ r_2 &= q_3 \cdot r_3 + r_4 \Rightarrow r_4 = r_2 - q_3 \cdot r_3 \\ &\vdots \\ r_{n-3} &= q_{n-2} \cdot r_{n-2} + r_{n-1} \Rightarrow r_{n-1} = r_{n-3} - q_{n-2} \cdot r_{n-2} \\ r_{n-2} &= q_{n-1} \cdot r_{n-1} + r_n \Rightarrow r_n = r_{n-2} - q_{n-1} \cdot r_{n-1} \\ r_{n-1} &= q_n \cdot r_n \Rightarrow r_n = \gcd(a, b) \end{aligned}$$

Then,

$$\begin{aligned} \gcd(a, b) &= r_n = \alpha_{n-1} r_{n-2} + \beta_{n-1} r_{n-1} \\ &= \alpha_{n-2} r_{n-3} + \beta_{n-2} r_{n-2} \\ &= \dots \\ &= \alpha_3 r_2 + \beta_3 r_3 \end{aligned}$$

$$\begin{aligned}
&= \alpha_2 b + \beta_2 r_2 \\
&= \alpha_1 a + \beta_1 b
\end{aligned}$$

□

Remark

Bézout's identity does not imply that the integers x and y are unique.

The identity allows us to redefine the gcd through the following theorem.

Theorem 3.19

Let a and b be two integers not simultaneously zero with $\gcd(a, b) = d$. An integer c can be written in the form $a \cdot x + b \cdot y$ for some integers x and y if and only if c is a multiple of d . In particular, the gcd d is the smallest positive integer of the form $a \cdot x + b \cdot y$ with $x, y \in \mathbb{Z}$.

And as a direct consequence, to redefine relatively-prime integers through the following corollary.

Corollary 3.19.1

Two integers are relatively prime if and only if there exist integers x and y such that

$$a \cdot x + b \cdot y = 1$$

Corollary 3.19.2

If $\gcd(a, b) = d$, then

- (i) $\forall m \in \mathbb{N}, \gcd(m \cdot a, m \cdot b) = m \cdot d$
- (ii) $\gcd\left(\frac{a}{d}, \frac{b}{d}\right) = 1$

Corollary 3.19.3

If a, b are two relatively-prime integers, then:

- (i) If $a \mid c$ and $b \mid c$, then $(a \cdot b) \mid c$.
- (ii) If $a \mid (b \cdot c)$, then $a \mid c$.

Theorem 3.20

Let m be a positive integer, and let a, b, c be integers. If $a \cdot c \equiv b \cdot c \pmod{m}$ and $\gcd(c, m) = 1$, then $a \equiv b \pmod{m}$.

Remark

- This theorem allows us to divide by a common factor c both sides of the sign $\equiv$ whenever c and the modulus m are relatively prime.
- If c and m are not relatively prime, then the correct result is: Let us write $m = p \cdot c$ for positive integers p, c , and let a, b be integers. If $a \cdot c \equiv b \cdot c \pmod{p \cdot c}$, then $a \equiv b \pmod{p}$.

3.4 Prime Numbers

3.4.1 Definition

Definition 3.21 (Prime numbers)

A natural number $p > 1$ is called a **prime number** if the only positive divisors of p are 1 and p . A natural number $p > 1$ that is not prime is called **composite**.

Remark

The natural number 1 is not prime. The first prime number is 2, and the other prime numbers are odd natural numbers (3, 5, 7, 11, ...).

Theorem 3.22 (Euclid)

There are infinitely many prime numbers.

3.4.2 Primes as building blocks

Theorem 3.23 (The Fundamental Theorem of Arithmetic)

Every natural number $n > 1$ can be written uniquely as a product of primes

$$n = p_1^{n_1} \cdot p_2^{n_2} \cdot p_3^{n_3} \cdot \dots \cdot p_k^{n_k}$$

where the p_i are distinct prime numbers written in increasing order, and the exponents n_i are natural numbers $n_i \geq 1$.

Theorem 3.24

The positive integer n is a composite number if and only if n can be divided by some prime number $p \leq \sqrt{n}$.

Lemma 3.25

Let p be a prime number, and let a, b be integers. Then:

- (i) Either $p \mid a$, or p and a are relatively prime.

(ii) If $p \mid (a \cdot b)$, then either $p \mid a$ or $p \mid b$.

Proposition 3.26

If the integers $a, b > 1$ can be factorized in the form

$$\begin{aligned} a &= p_1^{n_1} \cdot p_2^{n_2} \cdots p_k^{n_k} \\ b &= p_1^{m_1} \cdot p_2^{m_2} \cdots p_k^{m_k} \end{aligned}$$

with $n_i, m_i \geq 0$ and all prime factors of a and b appear in both decompositions, then

$$\begin{aligned} \gcd(a, b) &= p_1^{\min(n_1, m_1)} \cdot p_2^{\min(n_2, m_2)} \cdots p_k^{\min(n_k, m_k)} \\ \text{lcm}(a, b) &= p_1^{\max(n_1, m_1)} \cdot p_2^{\max(n_2, m_2)} \cdots p_k^{\max(n_k, m_k)} \end{aligned}$$

3.5 Linear Congruence Equations

Definition 3.27

A congruence modulo m of the form

$$a \cdot x \equiv b \pmod{m}$$

where m is a positive integer, a, b are integers, and x is a variable is called a linear congruence equation.

Remarks:

- If there exists a unique solution of the linear congruence equation $a \cdot x \equiv 1 \pmod{m}$, then solving this equation is equivalent to obtaining the multiplicative inverse of a modulo m .
- If x is a solution of a linear congruence equation, and $x' \equiv x \pmod{m}$, then x' is also a solution of that equation: $a \cdot x' \equiv a \cdot x \pmod{m} \equiv b \pmod{m}$.
- Therefore, the solutions of a linear congruence equation (if any) form classes of congruence modulo m : i.e., they are elements of $\mathbb{Z}_m$.

3.5.1 Solving Linear Congruence Equations

Theorem 3.28

If $d = \gcd(a, m)$, then the linear congruence equation

$$a \cdot x \equiv b \pmod{m}$$

has a solution if and only if $d \mid b$. In this case and if x_0 is a particular solution of the linear congruence equation, the general solution is given by

$$x_k = x_0 + (m \cdot k)/d, \quad k \in \mathbb{Z}$$

In particular, these solutions form d congruence classes modulo m with representatives: $\{x_0, x_0 + m/d, x_0 + (2m)/d, \dots, x_0 + (m(d-1))/d\}$

Corollary 3.28.1

If $\gcd(a, m) = 1$, the solutions x of the linear congruence equation $a \cdot x \equiv b \pmod{m}$ form a unique congruence class modulo m .

Corollary 3.28.2

If $\gcd(a, m) = 1$ with $m > 1$, then there exists a multiplicative inverse of a modulo m . This multiplicative inverse is unique modulo m .

3.5.2 Arithmetic with $\mathbb{Z}_m$

The elements of $\mathbb{Z}_m$ with $m \in \mathbb{N}$ are equivalence classes modulo m . For the sake of simplicity, $x \in \mathbb{Z}_m$ represents that $x \in [x]_m$.

The sum and the multiplication on $\mathbb{Z}_m$ are defined as:

$$\begin{aligned}x + y &= [x]_m + [y]_m = [x + y]_m \\x \cdot y &= [x]_m \cdot [y]_m = [x \cdot y]_m\end{aligned}$$

and they verify the usual properties: for every $x, y, z \in \mathbb{Z}_m$,

- Closure: $x + y \in \mathbb{Z}_m$ and $x \cdot y \in \mathbb{Z}_m$.
- Associativity: $x + (y + z) = (x + y) + z$ and $x \cdot (y \cdot z) = (x \cdot y) \cdot z$.
- Commutativity: $x + y = y + x$ and $x \cdot y = y \cdot x$.
- Distributivity: $x \cdot (y + z) = x \cdot y + x \cdot z$.
- Identity element (sum): $\exists 0 \in \mathbb{Z}_m$ such that $0 + x = x$, $\forall x \in \mathbb{Z}_m$.
- Identity element (product): $\exists 1 \in \mathbb{Z}_m$ such that $1 \cdot x = x$, $\forall x \in \mathbb{Z}_m$.
- Inverse element (sum): $\forall x \in \mathbb{Z}_m$, $\exists -x \in \mathbb{Z}_m$ such that $x + (-x) = 0$.

Remark: These properties are those characterizing a field [like $(\mathbb{R}, +, \cdot)$], except for the existence of a multiplicative inverse.

In $\mathbb{Z}$ there does not exist in general the multiplicative inverse of an integer x : y is the multiplicative inverse of x if and only if $x \cdot y = 1$. However, two properties hold:

- (i) Cancellation law: If $x \neq 0$ and $x \cdot y = x \cdot z$, then $y = z$.
- (ii) If $x \cdot y = 0$, then either $x = 0$ or $y = 0$.

None of these two properties holds in general in $\mathbb{Z}_m$.

Definition 3.29

An element $x \not\equiv 0 \pmod{m}$ of $\mathbb{Z}_m$ is a divisor of zero if there exists an element $y \not\equiv 0 \pmod{m}$ such that $x \cdot y \equiv 0 \pmod{m}$.

Remark: In some books, the condition $x \not\equiv 0 \pmod{m}$ is dropped.

Definition 3.30

An element $x \in \mathbb{Z}_m$ is a unit modulo m if it has a multiplicative inverse modulo m ; i.e., if there is an element $s \in \mathbb{Z}_m$ such that $x \cdot s \equiv 1 \pmod{m}$.

Theorem 3.31

The multiplicative inverse of a unit modulo m is unique.

Remark: As the inverse of a unit r modulo m is unique, it will be denoted by r^{-1} .

Theorem 3.32

An element $r \in \mathbb{Z}_m$ is invertible (i.e., it has a multiplicative inverse) if and only if r and m are relatively prime.

Corollary 3.32.1

If p is a prime number, every nonzero element of $\mathbb{Z}_p$ is invertible.

- If p is prime, then $(\mathbb{Z}_p, +, \cdot)$ is a field like $(\mathbb{R}, +, \cdot)$ or $(\mathbb{Q}, +, \cdot)$.
- If $m = p \cdot q$ is composite, then there are divisors of zero in $\mathbb{Z}_m$: $p \cdot q \equiv 0 \pmod{m}$ with $p, q \not\equiv 0 \pmod{m}$. In this case, $(\mathbb{Z}_m, +, \cdot)$ is a ring with divisors of zero.

Definition 3.33

Euler's (totient) function $\varphi : \mathbb{N} \rightarrow \mathbb{N}$ is defined as $\varphi(m)$ gives the number of invertible elements of $\mathbb{Z}_m$.

Lemma 3.34

If p is a prime number, then $\varphi(p) = p - 1$.

3.5.3 Euler's Theorem

Theorem 3.35 (Euler, 1790)

If y is invertible in $\mathbb{Z}_m$ (i.e., if $\gcd(y, m) = 1$), then

$$y^{\varphi(m)} \equiv 1 \pmod{m}$$

Corollary 3.35.1 (Fermat's Little Theorem)

If p is a prime number and if $y \not\equiv 0 \pmod{p}$, then

$$y^{p-1} \equiv 1 \pmod{p}$$

Corollary 3.35.2

If p is a prime number, then $y^p \equiv y \pmod{p}$ for any integer y .

Theorem 3.36

- (i) If p is a prime, then $\varphi(p^k) = p^{k-1}(p-1)$ for every $k \in \mathbb{N}$.
- (ii) If $\gcd(m, n) = 1$, then $\varphi(m \cdot n) = \varphi(m) \cdot \varphi(n)$.
- (iii) If $n \geq 2$ has the following decomposition in prime factors $n = \prod_{k=1}^r p_k^{n_k}$ with $n_k \geq 1$, then $\varphi(n) = n \cdot \prod_{k=1}^r (1 - 1/p_k)$.

3.6 Applications

3.6.1 Random number generators

3.6.2 Cryptography

3.6.3 Further resources

- [Cool video on prime numbers](#) (and cool channel in general!)

Counting

 This chapter overlaps with sections 6.1, 6.3, 6.4, 6.5 of Rosen.

The **goal of counting** is basically to determine the cardinality of certain finite sets. This is much more useful than it sounds like: a fundamental way to compute discrete probabilities is to count in how many ways a given event can occur, and divide it by the total number of possible alternatives.

Definition 4.1

Two sets A and B have the same cardinality if and only if there exists a bijective function $f : A \rightarrow B$.

Definition 4.2 (Countable sets)

A set that is either finite or has the same cardinality as the set $\mathbb{N}$ is called **countable**.

4.1 Basic counting principles

4.1.1 The product principle

Suppose that a meal can be broken down into a first and second course. Let's assume we have n_1 recipes for the first course and n_2 for the second. If any combination of first and second course is deemed acceptable, then we have $n_1 \cdot n_2$ ways to prepare a whole meal.

Remark

The condition above can be rephrased as “if the choice of second course is independent from the choice of first course”. “Independent” is a word you'll hear again in probability theory. This is no coincidence!

This is the idea behind the product principle stated below.

Proposition 4.3 (The basic product principle)

If A and B are two finite sets, then

$$|A \times B| = |A| \cdot |B|$$

This can be directly generalised to an arbitrary number of sets.

Proposition 4.4 (The generalised product principle)

If $A_1, A_2, \dots, A_m$ are finite sets, then

$$|A_1 \times A_2 \times \dots \times A_m| = |A_1| \cdot |A_2| \dots |A_m| = \prod_{k=1}^m |A_k|$$

Example. TODO

4.1.2 The sum principle

Let's consider the cuisines of two different countries. Let's assume a given ingredient can be prepared in n_1 recipes in the first, and in n_2 recipes in the second. If the two countries do not share a single recipe in common, then when one considers the two cuisines as a whole, they can prepare $n_1 + n_2$ different recipes.

Proposition 4.5 (The basic sum principle)

If A and B are two finite and disjoint sets ($A \cap B = \emptyset$), then

$$|A \cup B| = |A| + |B|$$

This can be directly generalised to an arbitrary number of sets.

Proposition 4.6 (The generalised sum principle)

If $A_1, A_2, \dots, A_m$ are a sequence of finite and pairwise disjoint sets $A_i \cap A_j = \emptyset$ for all $i \neq j$, then

$$|A_1 \cup A_2 \cup \dots \cup A_m| = |A_1| + |A_2| + \dots + |A_m| = \sum_{j=1}^m |A_j|$$

Example. TODO

4.1.3 The inclusion-exclusion principle

We can generalise the sum principle by relaxing the assumption we made that the two cuisines do not share a single recipe in common. So let's be more agnostic and consider the possibility that they might. This leads to the inclusion-exclusion principle.

Proposition 4.7 (The basic inclusion-exclusion principle)

If A and B are two finite sets, then

$$|A \cup B| = |A| + |B| - |A \cap B|$$

You may see this graphically by drawing a Venn diagram.

❓ Question

Now, what happens if we add another set C to the mix? Try to answer by drawing another diagram.

The generalisation is less straightforward than before, but as hinted by our iterative approach of adding one subset at a time, the following can be proven by induction.

Proposition 4.8 (The generalised inclusion-exclusion principle)

If $A_1, A_2, \dots, A_m$ are finite sets, then

$$\begin{aligned} |A_1 \cup A_2 \cup \dots \cup A_n| &= \sum_{1 \leq i \leq n} |A_i| \\ &\quad - \sum_{1 \leq i < j \leq n} |A_i \cap A_j| \\ &\quad + \sum_{1 \leq i < j < k \leq n} |A_i \cap A_j \cap A_k| \\ &\quad \dots \\ &\quad + (-1)^{n+1} |A_1 \cap A_2 \cap \dots \cap A_n| \end{aligned}$$

4.2 Combinatorics

4.2.1 Permutations

A permutation is basically a way in which elements of a set can be ordered.

Example. Let's consider the set containing the first four letters of the alphabet: $S = \{A, B, C, D\}$. In how many ways can we order them, or, in other words, in how many ways can we permute the elements of this set, or, even, how many permutations does this set have? To see how to answer that, let's think about the very process of ordering. We first choose the element that should come first: at this stage we have 4 different choices. Once this choice is made, we have 3 choices left for the second, then after that, 2 choices for the third, and finally, only one. By the product rule we thus have $4 \cdot 3 \cdot 2 \cdot 1 = 24$ ways to order these elements.

Proposition 4.9 (Permutations of n distinct objects)

Given n distinct objects, there are $n!$ distinct ordered arrangements (= permutations) of these objects.

Remark

For a set A , we can directly say that it has $|A|!$ permutations, since its elements are distinct by definition.

We involved the factorial, that we now properly define.

Definition 4.10 (Factorial)

For all $n \in \mathbb{N}$, we define the **factorial** of n as

$$n! = n \cdot (n-1) \cdot (n-2) \dots 2 \cdot 1 = \prod_{k=1}^n k$$

 Remark

By convention, $0! = 1$.

Let us generalise the concept of a permutation by also considering we want to form orderings of size $r \leq n$.

Definition 4.11 (Permutation)

Given a set of cardinality n and a natural number $r \leq n$, an r -permutation of elements of this set is an ordered arrangement of r elements of this set.

 Notation

The number of such arrangements is denoted $P(n, r)$.

The result above can then be directly extended, as we simply stop making choices after r steps.

Proposition 4.12

Given a set of cardinality n and a natural number $r \leq n$, the number of r -permutations we can form from this set is:

$$P(n, r) = n! \cdot (n - 1)! \cdot \dots \cdot (n - r + 1)! = \frac{n!}{(n - r)!}$$

 Remark

We can get back the result of [Proposition 4.9](#) for n -permutations using the convention $0! = 1$.

If we tweak the permutation process by considering that each choice is independent from the others, we're now allowed to pick any element from the set at every step, even those we've already picked before. We're thus allowing repetitions in our permutation, and the number of such permutations is even more straightforward to obtain.

Proposition 4.13

The number of r -permutations of a set of n objects with repetition allowed is n^r .

Now what happens if our objects are not necessarily distinguishable anymore? Let's say we now want to know in how many ways we can shuffle the string $ABBCCC$. If we attached labels to each letter to keep track of them, like such: $A_1 B_1 B_2 C_1 C_2 C_3$, then as above, we would have $(1 + 2 + 3)! = 6!$ permutations. But notice that if we remove the labels, the two following permutations

$$\begin{array}{c} A_1 B_1 B_2 C_1 C_2 C_3 \\ A_1 B_2 B_1 C_1 C_2 C_3 \end{array}$$

in which we permuted the B 's, would look exactly the same! So for each labeled permutation, two unlabeled ones would have the same ordering for the B 's. Similarly, the C 's can be reordered in $3!$ ways in each labeled permutation. Therefore, all in all, each labeled permutation has $2! \cdot 3! = 12$ equivalent unlabeled ones. We can thus get the number of ways we can shuffle this string as:

$$\frac{6!}{2! \cdot 3!} = \frac{6 \cdot 5 \cdot 4}{2} = 60$$

Proposition 4.14 (Permutations with indistinguishable objects)

Given n objects that can be classified into k groups of indistinguishable elements, and such that the first group contains n_1 indistinguishable elements of type 1, the second group contains n_2 indistinguishable elements of type 2, etc, then the number of distinct ordered arrangements of these objects is

$$\frac{n!}{n_1! n_2! \dots n_k!}$$

Remark

By definition of the n_i above, $\sum_{i=1}^k n_i = n$.

4.2.2 Combinations

What if we now count distinct unordered selections of objects from a set, that is, how many distinct subsets of the same size we can form? This is called a combination. If we simply combine all the elements from a set with n elements, by definition of a set there is only one such combination of n elements. What's interesting is if we wonder how many combinations of $r < n$ elements from this set of n elements we can form.

Definition 4.15 (Combination)

Given a set of cardinality n and $r \leq n$, an r -combination of elements of this set is one of its subsets of cardinality r .

Notation

The number of such subsets is denoted $C(n, r)$.

As we've seen above, a set of size n admits $\frac{n!}{(n-r)!}$ permutations of size r . But, once we have an r -permutation, this new arrangement itself can be permuted in $P(r, r) = r!$ ways, each of which corresponds to the same subset. That's how we can see that

Proposition 4.16

Given a set of cardinality n and a natural number $r \leq n$, the number of r -combinations we can form from this set is:

$$C(n, r) = \frac{P(n, r)}{r!} = \frac{n!}{r!(n-r)!}$$

Remark

Notice how this does give us $C(n, n) = 1$, using again the convention $0! = 1$.

This leads us to introduce some particular integers: the binomial coefficients.

Definition 4.17 (Binomial coefficients)

For all non-negative integers $n, r \in \mathbb{Z}^+$ such that $0 \leq r \leq n$, we define the **binomial coefficient** as follows:

$$\binom{n}{r} = \frac{n!}{r!(n-r)!}$$

Question

Rewrite $C(n, r)$ and $P(n, r)$ using a binomial coefficient.

Remark

The symbol $\binom{n}{r}$ is read “ n choose r ”, as it gives us the number of ways we can choose r distinct elements from a set of size n . Forming an r -combination is thus equivalent to sampling without replacement, to translate our combinatorics approach to the language of statistics.

4.2.3 Distributions

Many counting problems can be solved by mapping them to a process of distributing objects into boxes. Same as above, the objects can be considered distinguishable, or labeled. The distinction could also be made for the boxes, but in this course we will only consider distinguishable boxes.

Let's first consider n distinguishable objects to be placed into k boxes through an example. *Example.* How many ways are there to distribute hands of 5 cards to each of four players from a deck of 52 cards? This problem does correspond to the general one stated above, because all cards and players are distinct. We can notice that once we dealt cards to a player i , it is as if we created a group of $n_i = 5$ indistinguishable cards, because once in their hands, the order in which they arrived does not matter, only which cards they received. We thus create

one of these groups for each of the four players, and a final group with the remaining $52 - 4 \cdot 5 = 32$ cards. This amounts to creating $k = 5$ groups, each of which contains n_i elements of the same type. We can thus form a bijective function between the distributions of cards to the players and permutations with indistinguishable objects from [Proposition 4.14](#)! From [Definition 4.1](#), we can therefore use the previous result to get that the total number of ways to distribute the hands is

$$\frac{52!}{4 \cdot 5!32!}$$

We thus get the same result as in [Proposition 4.14](#).

Proposition 4.18 (Distributions of distinguishable objects)

The number of distributions of n distinguishable objects into k distinguishable boxes so that each box i receives n_i objects is

$$\frac{n!}{n_1!n_2!\dots n_k!}$$

Let's now consider we have n indistinguishable objects. Let's use the stars and bars methods to derive the corresponding count. Since objects are indistinguishable, they can be represented by any same symbol, such as a star $*$, which is conventionally used as a placeholder. The assignment to k boxes can then be represented by separating these stars with $k - 1$ bars. *Example.* For $n = 8$ and $k = 3$, two example distributions can thus be visualised as:

$$\begin{array}{c} *** \mid **** \mid * \\ A \quad B \quad C \end{array} \quad \begin{array}{c} \mid **** \mid **** \\ A \quad B \quad C \end{array}$$

So the $n + k - 1 = 10$ stars and bars are to be arranged in as many slots, so we may for instance choose $k - 1$ of these to contain bars, and then fill the rest with stars.

This is thus a “ $k - 1$ choose $n + k - 1$ ” combination!

Proposition 4.19 (Distributions of indistinguishable objects)

The number of distributions of n indistinguishable objects into k distinguishable boxes is

$$\binom{n + k - 1}{k - 1}$$

Another parallel with combinations can be established. Counting the number of ways of placing n indistinguishable objects into k boxes turns out to be the same as counting the number of n -combinations for a set with k elements when repetitions are allowed. These are slightly different from [Definition 4.15](#), since we allow to redraw the same element from the input set. The combination with repetition is thus not a subset, but a collection of elements with potential repetitions. The processes behind the distribution and the combination with repetition can be directly mapped by imagining that each time the i^{th} element of the input set is included in the combination, we put a ball in the i^{th} box of the distribution.

Proposition 4.20 (r -combinations with repetition)

Given a set of cardinality n and a natural number $r \leq n$, the number of r -combinations with repetition that we can form from this set is:

$$\binom{n+r-1}{n-1}$$

4.2.4 Binomial coefficients

There are many useful identities related to these coefficients. One of the most important is Pascal's identity.

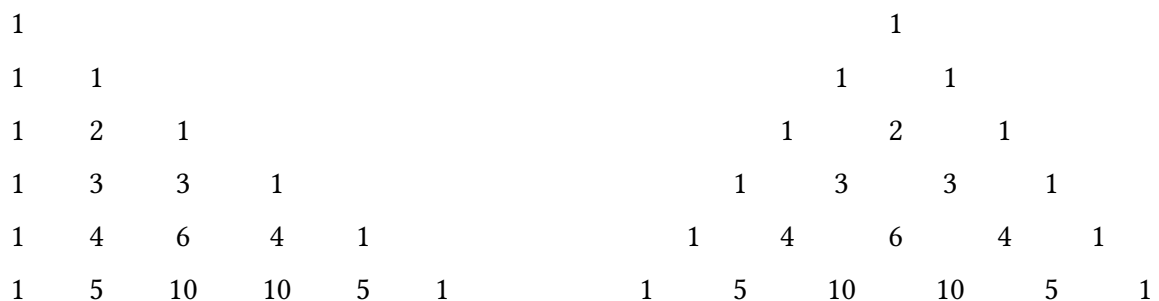
Theorem 4.21 (Pascal's identity)

$$\forall r \in \mathbb{Z}^+, n \geq r, \binom{n+1}{r} = \binom{n}{r} + \binom{n}{r-1}$$

Using this identity, the binomial coefficients can be visualised by arranging them such that

$$\binom{n}{r} \equiv \begin{pmatrix} \text{row number} \\ \text{column number} \end{pmatrix}$$

Starting with the initial conditions $\binom{n}{0} = \binom{n}{n} = 1$, we thus form Pascal's triangle, expanding it downwards using [Theorem 4.21](#). Pascal's triangle can then be drawn left or centre-aligned, as preferred:



This also allows us to see the symmetry in the binomial coefficients, especially using the centre-aligned representation.

Theorem 4.22 (Symmetry)

$$\forall r \in \mathbb{N}, n \geq r, \binom{n}{r} = \binom{n}{n-r} = \frac{n!}{r!(n-r)!}$$

The binomial coefficients also appear in the expansion of some polynomials.

Example. Let's take the example of the expansion of $(x + y)^3$. Its expansion can be seen as a combinatorics problem in which we repeatedly pick either x or y from each parenthesised term of $(x + y)^3 = (x + y)(x + y)(x + y)$ to form a product. This gives us 2^3 products of the form $x^\alpha y^\beta$ with $0 \leq \alpha, \beta \leq 3$, which we then sum. We can thus simply count the number of ways to choose α times x (then $\beta = 3 - \alpha$) from the three parenthesised terms, which is $\binom{3}{\alpha}!$ Explicitly, here:

$$(x + y)^3 = xxx + xxy + xyx + xyy + yxx + yxy + yyx + yyy,$$

so the number of ways we can put two x in three slots is $\binom{3}{2}$, to put only 1 is $\binom{3}{1}$ and only $\binom{3}{3} = \binom{3}{0} = 1$ way to put either three or zero of them.

This leads to the binomial theorem.

Theorem 4.23 (Newton's binomial theorem)

Let x and y be two variables, and $n \in \mathbb{N}$. Then

$$(x + y)^n = \sum_{k=0}^n \binom{n}{k} x^k y^{n-k}$$

Corollary 4.23.1

$$(1 + x)^n = \sum_{k=0}^n \binom{n}{k} x^k$$

Corollary 4.23.2

$$\sum_{k=0}^n \binom{n}{k} = 2^n, \quad \sum_{k=0}^n (-1)^k \binom{n}{k} = 0$$

Finally, the binomial theorem together with [Proposition 4.16](#) enables us to determine the cardinality of the power set.

Corollary 4.23.3

Given a finite set A of cardinality n ,

$$|\mathcal{P}(A)| = \sum_{r=0}^n C(n, r) = 2^n$$

 Remark

You will find again the binomial coefficients in the binomial distribution and other distributions in probability theory.

Sequences

 This chapter overlaps with sections 2.4, 8.2, 8.4 of Rosen.

5.1 Defining sequences

Sequences are basically indexed collections of elements, or, to be more formal:

Definition 5.1 (Sequence)

A sequence is a function from a set of indices $I \subseteq \mathbb{N}$ to a set A of elements at each index. The image in A of the integer $n \in I$ is denoted a_n , and is called the n^{th} term of the sequence. The sequence can be represented as $(a_n)_{n \in I}$.

Remark

There are many ways to represent sequences in computers. In  Python for instance, there are bytes (bytes sequences), str (text sequences), list, tuple, or even range which [are sequence types](#). Note how [even in !\[\]\(3d842d2b9313bc6ab5f4e186d8bedd33_img.jpg\) Python](#) the difference is made between sequences and sets: the Sequence's central method is `__getitem__`—which allows you to retrieve their n th element—, which doesn't exist for Set, whose central method is `__contains__`, to check if an element is inside the set.

The terms of a sequence can be defined explicitly, with a closed form...

Example. Let's define the sequence $(a_n)_{n \in \mathbb{N}}$ where $a_n = 2^n$. Then the terms of the sequence are 1, 2, 4, 8, ...

... but not only! They may also be defined recursively, that is, providing some initial terms together with a rule that is needed to determine subsequent terms.

Definition 5.2 (Recurrence relation)

A **recurrence relation** for the sequence $(a_n)_{n \in \mathbb{N}}$ is an equation that expresses a_n in terms of one or more previous terms $a_{n-1}, a_{n-2}, \dots, a_{n-k}$, and possibly n . The **initial conditions** are the first k terms in the sequence $(a_1, \dots, a_k)$, which are necessary to be able to determine all terms of such a sequence.

Example. Redefine the previous sequence ($a_n = 2^n$) and the factorial ($b_n = n!$) using recurrence relations.

Remark

- More than one sequence may satisfy the same recurrence relation, hence the importance of initial conditions.
- From the principle of strong induction ([Theorem 2.22](#)), we know that a recurrence relation together with a set of initial conditions define a unique sequence.

5.2 Aggregating sequences

Sometimes, we're not interested in every term of the sequence, but in some aggregate. For instance, you're not really interested in all movements in your bank account, but in your balance, so the sum of all these movements.

The two fundamental ways to aggregate a sequence are the summation and products, that we introduce below.

Definition 5.3 (Sequence sum)

Given a sequence $(a_n)_{n \in I}$ with $I \subseteq \mathbb{N}$, its **sum** is the sum of its terms. Very often, we sum over contiguous integers, so $I = \{m, m+1, \dots, M\}$ with $M > m$, and we write it

$$\sum_{n \in I} a_n = a_m + a_{m+1} + \dots + a_M = \sum_{n=m}^M a_n.$$

If I is infinite, the sum is called a **series**. If I is empty, the sum is 0.

Definition 5.4 (Sequence product)

Given a sequence $(a_n)_{n \in I}$ with $I \subseteq \mathbb{N}$, its **product** is the product of its terms. Very often, we sum over contiguous integers, so $I = \{m, m+1, \dots, M\}$ with $M > m$, and we write it

$$\prod_{n \in I} a_n = a_m a_{m+1} \dots a_M = \prod_{n=m}^M a_n.$$

If I is infinite, the sum is called an **infinite product**. If I is empty, the product is 1.

The usual laws of arithmetic apply to these sums and products.

Proposition 5.5

Let (x_n) and (y_n) be two sequences, $I \subseteq \mathbb{N}$ and $a, b \in \mathbb{R}$, then

$$\sum_{i \in I} (ax_n + by_n) = a \sum_{i \in I} x_n + b \sum_{i \in I} y_n$$

$$\prod_{i \in I} (x_n^a y_n^b) = \left(\prod_{i \in I} x_n \right)^a \left(\prod_{i \in I} y_n \right)^b$$

We can often prove a closed form for a sum or a product using induction. To show it, let's introduce two fundamental kinds of sequences.

Definition 5.6 (Arithmetic progression)

An arithmetic progression is a sequence $(a_n)_{n \in \mathbb{N}}$ that satisfies the recurrence relation

$$a_{n+1} = a_n + d$$

where $d \in \mathbb{R}$ is called the common difference.

Definition 5.7 (Geometric progression)

A geometric progression is a sequence $(a_n)_{n \in \mathbb{N}}$ that satisfies the recurrence relation

$$a_{n+1} = ra_n$$

where $r \in \mathbb{R}$ is called the common ratio.

Then the following can be proven by induction.

Proposition 5.8

For all $n \in \mathbb{N}$ and $r \in \mathbb{R} \setminus \{1\}$:

$$\begin{aligned} \text{(i)} \quad \sum_{k=0}^n k &= \frac{n(n+1)}{2} \\ \text{(ii)} \quad \sum_{k=0}^n r^k &= \frac{1-r^{n+1}}{1-r} \end{aligned}$$

5.3 Solving recurrence relations

We have seen above how to translate an explicit sequence definition into a recurrence relation. However, most of the time, it's the other way around. In this case, we want to **solve** the recurrence relation, that is, to find the equivalent explicit sequence definition, also known as a **closed formula**, if it exists.

5.3.1 Linear homogeneous recurrence relations

Let us first study linear recurrence relations, as they often appear in real-world problems, and can be solved systemically.

Definition 5.9

- A recurrence relation is of k^{th} **order** if a_n can be expressed in terms of its k previous terms $a_{n-1}, a_{n-2}, \dots, a_{n-k}$.
- A recurrence relation is **linear** if it expresses a_n as a linear combination of its previous terms.
- A recurrence relation is **homogeneous** if the zero sequence $a_n = 0$ satisfies the relation.

Proposition 5.10

A k^{th} order linear recurrence relation with constant coefficients is of the form

$$a_n = c_1 a_{n-1} + c_2 a_{n-2} + \dots + c_k a_{n-k} + f(n)$$

where $c_1, \dots, c_k \in \mathbb{R}$, $c_k \neq 0$, and $f : \mathbb{N} \rightarrow \mathbb{R}$ which does not depend on any sequence term. The relation is also homogeneous if and only if $f(n) = 0$ for all n .

To gain some intuition on the solution of such recurrence relations, let's start simple.

Theorem 5.11 (Solution of a homogeneous first-order recurrence relation)

Let us suppose that the sequence $(a_n)_{n \in \mathbb{N}}$ satisfies the recurrence relation

$$\forall n \geq 1, a_n = ca_{n-1},$$

where $c \in \mathbb{R}$, and we know the initial condition a_1 . Then, the solution of this relation is given by

$$\forall n \in \mathbb{N}, a_n = a_0 c^n.$$

Let us now observe that $a_n = \alpha x^n$ with $\alpha, x \in \mathbb{R}^*$ is a solution of such recurrence relations if and only if

$$\begin{aligned} \alpha x^n &= c_1 \alpha x^{n-1} + c_2 \alpha x^{n-2} + \dots + c_k \alpha x^{n-k} \\ \Leftrightarrow x^k - c_1 x^{k-1} - c_2 x^{k-2} - \dots - c_{k-1} x - c_k &= 0 \\ \Leftrightarrow P(x) &= 0 \end{aligned}$$

where we introduced the **characteristic polynomial** P , whose roots we will call the **characteristic roots**. The equivalence above implies that if x_i is a characteristic root, αx_i^n is a solution of the recurrence relation. The multiplicative α cancelled out above, so its value is to be fixed according to the initial conditions. The characteristic polynomial of a k^{th} order linear recurrence is of degree k , so it admits k characteristic roots, $x_1, \dots, x_k$.

Let us first suppose all roots are distinct. Then the solution is a linear combination of the form:

$$a_n = \sum_{i=1}^k \alpha_i x_i^n$$

Let us now suppose we have a root x_i of multiplicity $k_i > 1$. Then the derivative P' of the characteristic polynomial also has x_i as a root, of multiplicity $k_i - 1$! So x_i also satisfies

$$\begin{aligned} P'(x_i) &= 0 \\ \Leftrightarrow kx_i^{k-1} - c_1(k-1)x_i^{k-2} - \dots - c_{k-1} &= 0 \\ \Leftrightarrow kx_i^k - c_1(k-1)x_i^{k-1} - \dots - c_{k-1}x_i &= 0 \end{aligned}$$

which shows that nx_i^n is also a solution. Thus, the fact that x_i is a root of the first, second, ..., $(k-1)^{\text{th}}$ derivative of the characteristic polynomial P enables us to see that $nx_i^n, n^2x_i^n, \dots, n^{k-1}x_i^n$ are also solutions.

Putting everything together, we get the following general result.

Theorem 5.12 (Solution of a linear homogeneous recurrence relation)

Let (a_n) be a sequence satisfying a recurrence relation of the form:

$$a_n = c_1 a_{n-1} + c_2 a_{n-2} + \dots + c_k a_{n-k}$$

with $c_1, \dots, c_k \in \mathbb{R}$. Let us assume that its characteristic polynomial

$$P(x) = x^k - c_1 x^{k-1} - c_2 x^{k-2} - \dots - c_{k-1} x - c_k$$

has r distinct roots $x_1, \dots, x_r$ with multiplicities $k_1, \dots, k_r$.

Then, a sequence (a_n) is a solution of the recurrence relation if and only if

$$\forall n \in \mathbb{N}, a_n = \sum_{i=1}^r \left(\sum_{j=0}^{m_i-1} \alpha_{i,j} n^j \right) x_i^n$$

where $\alpha_{i,j}$ are constants to be fixed according to initial conditions.

Let us see what this gives in practice with Fibonacci-like recurrences.

Corollary 5.12.1 (Solution of a homogeneous Fibonacci-like recurrence relation)

Let us suppose that the sequence $(a_n)_{n \in \mathbb{N}}$ satisfies the recurrence relation

$$\forall n \geq 2, a_n = c_1 a_{n-1} + c_2 a_{n-2}$$

with $c_1, c_2 \in \mathbb{R}$. If the **characteristic equation** associated to this relation

$$x^2 = c_1 x + c_2$$

has characteristic roots x_1 and x_2 , then the solution of the recurrence relation is

$$\forall n \in \mathbb{N}, a_n = \begin{cases} \alpha_1 x_1^n + \alpha_2 x_2^n & \text{if } x_1 \neq x_2 \\ (\alpha_1 + n\alpha_2)x_1^n & \text{if } x_1 = x_2 \end{cases}$$

where the constants α_1 and α_2 can be obtained using the initial conditions a_1, a_2 .

Example. TODO with actual Fibo

5.3.2 Linear nonhomogeneous recurrence relation

What if we now add heterogeneity?

Theorem 5.13 (Solution of a linear nonhomogeneous recurrence relation)

Let us consider the linear nonhomogeneous recurrence relation with constant coefficients:

$$a_n = c_1 a_{n-1} + c_2 a_{n-2} + \dots + c_k a_{n-k} + f(n)$$

where $c_1, c_2, \dots, c_k \in \mathbb{R}$, and the initial conditions $a_1, \dots, a_k$ are known.

Then, the general solution of this linear nonhomogeneous recurrence is equal to

$(a_n^{(h)} + a_n^{(p)})_{n \in \mathbb{N}}$, where $(a_n^{(h)})$ is the solution of the associated *homogeneous* recurrence relation

$$a_n = c_1 a_{n-1} + c_2 a_{n-2} + \dots + c_k a_{n-k},$$

and $(a_n^{(p)})$ is any *particular* solution of the nonhomogeneous relation.

So the hard part is to “guess” a particular solution of the nonhomogeneous relation. In some simple examples though, it is not too hard to make the right guess.

Example. TODO, like example 10 p 548

5.4 Generating functions

5.4.1 Definitions

We will now introduce generating functions, which are a very powerful tool to manipulate sequences. The idea is not to find a function which is “equal” to the sequence in any sense, but one that **encodes the sequence**. To be more specific, when you formulate a generating function you compress all the information necessary to reproduce the full sequence, so the value of all its terms, into a simple function such as $f(x) = e^x$, $f(x) = \frac{1}{1-x}$, etc. How can a function contain all this information, though? By using the fact that many functions can be written as a power series, which means there is a sequence $(a_n)_{n \in \mathbb{N}}$ such that

$$f(x) = \sum_{n=0}^{+\infty} a_n x^n.$$

An obvious example is the polynomials: for instance $P(x) = 1 + 2x^2$ can be written as a power series with sequence (a_n) such that $a_0 = 1$, $a_2 = 2$, and $\forall n \in \mathbb{N} \setminus \{0, 2\}, a_n = 0$. But many other functions can be written as a power series, for instance by using their Taylor expansion! For instance, $e^x = \sum_{n=0}^{+\infty} \frac{x^n}{n!}$, so its associated sequence is $a_n = \frac{1}{n!}$. We can thus say that e^x *generates* this sequence, in the sense that it can tell you the values of all its terms. Contrary to what you’ve seen previously in calculus though, here we do things the other way around: we start from a sequence and try to find its generating function.

Definition 5.14

The **generating function** (GF) associated to the sequence $(a_n)_{n \in \mathbb{N}}$ is a function G which can be written as a formal power series with the terms of the sequence as coefficients:

$$G(x) = a_0 + a_1 x + a_2 x^2 + \dots + a_n x^n + \dots = \sum_{n=0}^{+\infty} a_n x^n$$

The series itself is called a **generating series**.

 Remark

We used the term *formal* because the series is only used as a representation of the sequence. x shouldn't be seen as an actual variable taking real values, we only use x^n as a symbol that marks, or corresponds to "where" the n^{th} term of the sequence is in the series. It is only when we state the equality of a generating series with a function that the associated "real" series should be well-defined, that is, it should converge for at least some values of x .

Because we have this abstraction of a formal power series, we can redefine how to add or multiply them together.

Definition 5.15

Let $f(x) = \sum_{n=0}^{+\infty} a_n x^n$ and $g(x) = \sum_{n=0}^{+\infty} b_n x^n$. Then

$$(i) \quad f(x) + g(x) = \sum_{n=0}^{+\infty} (a_n + b_n) x^n, \quad (\text{addition})$$

$$(ii) \quad f(x)g(x) = \sum_{n=0}^{+\infty} \left(\sum_{j=0}^k a_j b_{k-j} \right) x^n, \quad (\text{multiplication})$$

$$(iii) \quad x^m f(x) = \sum_{n=m}^{+\infty} a_{n-m} x^n \quad (\text{shifting})$$

 Remark

This actually matches the algebra of actual power series, simply because we defined it so it matches, since we want to eventually interpret the series as a closed-form function.

A summary of known series which can be identified when determining a closed form from a generating series is shown in Table 5.1.

Power series	Closed form	Reference
$\sum_{n=0}^k \binom{k}{n} x^n$	$(1+x)^k$	Theorem 4.23
$\sum_{n=0}^k x^n$	$\frac{1-x^{k+1}}{1-x}$	Proposition 5.8
$\sum_{n=0}^{+\infty} x^n$	$\frac{1}{1-x}$	Taylor
$\sum_{n=0}^{+\infty} \frac{x^n}{n!}$	e^x	Taylor
$\sum_{n=1}^{+\infty} \frac{(-1)^{n+1}}{n} x^n$	$\log(1+x)$	Taylor

Table 5.1: Some useful closed forms for power series

5.4.2 Solving recurrence relations

An example of the power of generating functions is how they can be used to solve recurrence relations, using the following procedure:

- (i) Rewrite the recurrence relation for a_n in terms of an equation that only involves the generating function G .
- (ii) Solve this equation and obtain a closed form for $G(x)$.
- (iii) Compute the coefficients a_n by performing the Taylor expansion of G around $x = 0$.

Example. Solve the recurrence relation $\forall n \in \mathbb{N}^*, a_n = 3a_{n-1}$, with initial condition $a_0 = 2$.

5.4.3 Solving counting problems

A generating series can encode a specific kind of sequence: namely, the sequence (a_n) of number of ways to select some object n times from a set. So for instance, the series $(1 + 2x + 3x^2)$ can encode the fact that we have a single way to not select the object, two to select it once, and three to select it three times.

Example. Let's see this with the following example: let's say we want to determine the number of ways to insert tokens worth 1€, 2€ and 5€ into a vending machine to pay for an item that costs r euros, when the order in which the tokens are inserted does not matter.

The number of euros produced by picking n tokens worth i euros can be seen as the exponent in $(x^i)^n = x^{in}$ within the formalism of generating functions. Besides, we might pick any number of tokens of each type, so for each type we need to sum over all possible number of picks n . Then the answer is the coefficient in front of x^r in the following generating function:

$$\left(\sum_{n=0}^{+\infty} x^n\right) \left(\sum_{n=0}^{+\infty} x^{2n}\right) \left(\sum_{n=0}^{+\infty} x^{5n}\right)$$

using the product rule. The independence of picks (so their order) is clearly visible in the generating function.

When the order matters, then using the product rule, the process of picking n tokens successively can be represented by $(x + x^2 + x^5)^n$. The number of ways for these n picks to sum up to r euros is then the coefficient of x^r . Since any number of tokens may be inserted, the number of ways to produce r euros when the order matters is the coefficient of x^r when we sum over all possible n values:

$$\sum_{n=0}^{+\infty} (x + x^2 + x^5)^n$$

In both cases, the answer might seem complicated to get by hand for large r , but it is relatively straightforward using a computer.

Remark

In generating functions representing combinatorial problems, you can think of additions as logical “or”, and products as logical “and”.

Graph theory

☞ This chapter overlaps with sections 10.1-6, 11.1 and 11.4-5 of Rosen.

6.1 Defining graphs

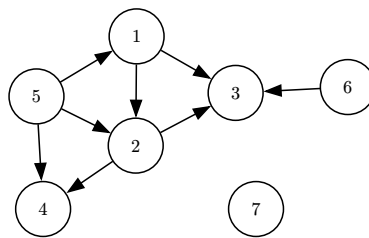
6.1.1 Basic definitions

Definition 6.1 (Directed graphs)

A **directed graph** is an ordered pair $G = (V, E)$ where

- V is a nonempty set of **vertices**, also called nodes.
- $E \subseteq V \times V$ is the graph's adjacency relation, which is a binary relation on V . Its elements are called **edges**, or links. If $e = (u, v) \in E$, we say that e is **incident** on u and v , that the two vertices are **adjacent**, that u is e 's **tail** and v its **head**, and also that u is v 's **predecessor** and v is u 's **successor**.

We actually already introduced directed graphs as a natural representation of a binary relation, in Section 1.4! Directed graphs can then be represented in a similar way. For instance:



Definition 6.2 (Undirected graphs)

An **undirected graph** is an ordered pair $G = (V, E)$ of vertex and edge sets V and E , where E is a set of *unordered* pairs of vertices, that is $E \subseteq \{\{u, v\} \mid u, v \in V\}$. If $e = \{u, v\} \in E$, we say that e is **incident** on u and v , or equivalently, that the two vertices are **adjacent**, or that u is in v 's **neighbourhood** $\mathcal{N}(v)$, and vice versa.

☞ Remark

The edge set of a directed graph is defined as

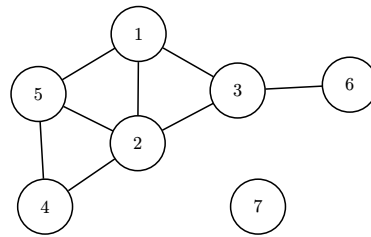
$$E \subseteq V \times V = \{(u, v) \mid u, v \in V\},$$

which is a set of *ordered* pairs of vertices.

☞ Remark

An undirected graph can be seen as a directed graph whose adjacency relation is symmetric —meaning if (u, v) is an edge, then (v, u) as well—, but also crucially considering that these are not two distinct edges but a single one, which can be represented as a 2-set $\{u, v\}$.

The representation of undirected graphs then simplifies the edges by removing the arrows indicating direction:



In the following, we'll use the word “**graph**” to refer to either a directed or undirected graph. Again in a similar fashion to relations, graphs can be represented by an adjacency matrix.

Definition 6.3 (Adjacency matrix)

Let's consider a graph $G = (V, E)$, and the ordering $v_1, v_2, \dots, v_{|V|}$ of its vertex set V . The **adjacency matrix** of G associated to that ordering is the $|V| \times |V|$ matrix whose entries A_{ij} count the number of edges between v_i and v_j .

Property 6.4

The adjacency matrix of an undirected graph is symmetric.

Notation

In the following, whenever we refer to a vertex as the vertex i or j , we mean v_i or v_j in the ordering $v_1, v_2, \dots, v_{|V|}$, from which the graph's adjacency matrix has been defined.

6.1.2 Particular graphs

An interesting particular case of graphs is the one that represents relationships between two separate sets of entities.

Definition 6.5 (Bipartite graphs)

A graph $G = (V, E)$ is **bipartite** if its vertex set V can be partitioned into two disjoint subsets V_1 and V_2 such that every edge in the graph connects a vertex in V_1 with a vertex in V_2 .

Remark

A directed bipartite graph can actually represent a binary relations between two sets.

We will now introduce some particular kinds of graphs which are not so common and that we will largely ignore in the following.

Definition 6.6 (Self-loops)

In a graph $G = (V, E)$, a vertex which is adjacent to itself is called a self-loop.

Remark

A graph can either allow or disallow self-loops, depending on what it models. Following the binary relation analogy, disallowing self-loops can be thought of as forcing the adjacency relation to be irreflexive.

The entries of the adjacency matrix defined in [Definition 6.3](#) have been defined as edge *counts*, meaning it could take values in $\mathbb{N}$, while graph definitions above implied values in $\{0, 1\}$, encoding the absence or presence of an edge. Actually, we defined adjacency matrices this way so it's general enough to account for multigraphs.

Definition 6.7 (Multigraphs)

A **multigraph** $G = (V, E)$ is a graph in which multiple edges are allowed between the same pair of vertices, which implies that E is not defined as a set but as a collection with potential repetitions.

Multigraphs are actually very rare in practice, as they can often be equivalently represented by weighted graphs.

Definition 6.8 (Weighted graphs)

A **weighted graph** $G = (V, E, \omega)$ is a graph such that every edge $e \in E$ is associated to a weight $\omega(e) \in \mathbb{R}$.

Remark

Naturally then, the entries of the adjacency matrix of a weighted graph correspond to the edge weights, that is:

$$A_{ij} = w((v_i, v_j)) = w_{ij}$$

Most of the time in this course, we will ignore these generalisations and only consider simple graphs.

Definition 6.9 (Simple graphs)

A simple graph is an unweighted graph without multi-edges or self-loops.

Definition 6.10 (Subgraphs)

The graph $H = (W, F)$ is a **subgraph** of $G = (V, E)$ if its vertex and edge sets are subsets of G 's, that is $W \subseteq V$ and $F \subseteq E$.

6.1.3 Isomorphism of graphs

The same graph can be represented in a number of ways: graphically the vertices are placed arbitrarily on a plane, and in an adjacency matrix, the ordering of vertices is also arbitrary. Also, if the vertex labels do not carry any particular meaning —i.e. they are “dummy” labels —, the labels too are arbitrary. It is thus important to be able to recognize the same graph, however it was represented.

Definition 6.11

The simple graphs $G_1 = (V_1, E_1)$ and $G_2 = (V_2, E_2)$ are **isomorphic** if and only if there exist a bijective function $f : V_1 \rightarrow V_2$ with the following property: a and b are adjacent in G_1 if and only if $f(a)$ and $f(b)$ are adjacent in G_2 . The function f is called an **isomorphism**.

Some simple tests based on aggregate measures can be used to discard isomorphism between two graphs.

Remark

Given two simple graphs $G_1 = (V_1, E_1)$ and $G_2 = (V_2, E_2)$, then

- (i) If $|V_1| \neq |V_2|$, then G_1 and G_2 are not isomorphic.
- (ii) If $|E_1| \neq |E_2|$, then G_1 and G_2 are not isomorphic.

Remark

G_1 and G_2 are isomorphic if there exists an invertible linear map (basically a permutation of the “basis vectors”) $\pi : V_1 \rightarrow V_2$ such that $A_2 = P^{-1} \cdot A_1 \cdot P$. There are $|V_1|! = |V_2|!$ maps of this type, hence the difficulty of testing for isomorphism if two graphs share many similarities at the aggregate level!

6.2 Traversing graphs

The whole point of a graph model is not to only look at binary relationships between pairs of entities represented by edges between pairs of vertices, but to investigate the interconnections over the whole set of vertices, thus including “long-distance relationships”. That is why traversing graphs is crucial, as it can provide us with central information about a graph.

6.2.1 Definitions

Definition 6.12

A **walk** on a graph $G = (V, E)$ is an alternating sequence of vertices and edges of the form $v_0, e_1, v_1, e_2, v_2, \dots, v_{l-1}, e_l, v_l$, such that e_k is from v_{k-1} to v_k for all k . It is said to be **closed** if it ends where it starts, so if $v_1 = v_l$, and open otherwise. The **length** of the walk is equal to the number of edges in the walk l , and is at least one.

Remark

- In an undirected graph, the condition on each edge is $e_k = \{v_{k-1}, v_k\}$, meaning that edges can be traversed in both directions.
- A walk on a graph without multi-edges can be described more simply by a sequence of vertices, as each edge is uniquely determined by a pair of vertices.

Definition 6.13

A **trail** is a walk with no repeated edge. A closed trail is called a **circuit**.

A **path** is a trail with no repeated vertex, with the potential exception of the first which may also appear as the last one. A **cycle** is a closed path.

Remark

The definitions of trails and paths are equivalent on a simple graph. In this case, we usually prefer to use the term “path”.

6.2.2 Graph connectivity

These definitions allow us to give a basic yet fundamental characteristic of a graph, namely, its connectivity.

Definition 6.14

An undirected graph is said to be **connected** if every pair of vertices is connected by a path.

Definition 6.15

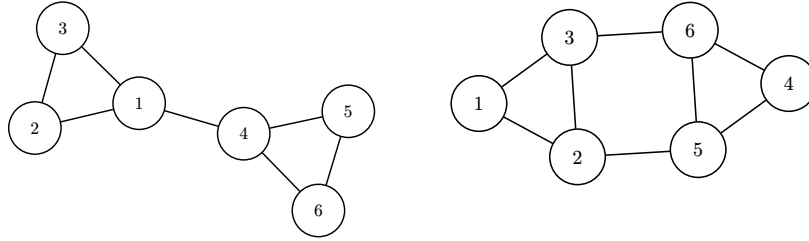
A directed graph $G = (V, E)$ is

- **strongly connected** if for every $u, v \in V$, there is a path from u to v and from v to u ,
- **weakly connected** if for every $u, v \in V$, there is a path between u and v in the underlying undirected graph.

A strongly or weakly disconnected graph can then be partitioned into strongly and weakly connected components.

? Question

The connectivity only tells us whether a graph is connected, but not how strongly it is. But given the following two graphs, which would you say is more strongly connected? Why?



One way to quantify the connectivity is to check how easy it is to split a graph into separate components.

Definition 6.16

A **cut edge** or **bridge** of a graph G is an edge whose removal produces a graph with more connected components than in G .

A **cut vertex** or **articulation point** of a graph G is a vertex whose removal (together with those edges incident with it) produces a graph with more connected components than in G .

Counting the minimum number of cut edges/vertices necessary to create k components thus allows a better characterisation of a graph's connectivity.

6.2.3 Vertices connectivity

How well two vertices are connected can then be measured by counting the number of walks which connect them.

Theorem 6.17

Let G be a graph with adjacency matrix A with respect to the ordering $v_1, v_2, \dots, v_{|V|}$ of its vertex set. The number of distinct oriented walks of length $n \geq 1$ that start at v_i and end at v_j is given by the entry (i, j) of the matrix A^n .

Corollary 6.17.1

Let G be a simple graph with adjacency matrix A , then

- $A_{ii}^2 = k(i)$ for every $1 \leq i \leq |V|$.
- $\text{tr } A^2 = 2|E|$.
- $\text{tr } A^3 = 6 \times \text{number of unoriented triangles in } G$.

6.2.4 Shortest paths

A very common problem related to paths is to find the shortest way to go from one vertex to another. But first, what does “short” mean here? Let’s generalise the notion of path length to weighted graphs.

Definition 6.18

Given a simple, undirected and weighted graph $G = (V, E, \omega)$ with positive weights, the length $L(P)$ of a path $P = v_0, e_1 v_1, e_2, v_2, \dots, v_{l-1}, e_l, v_l$ is the sum of the weights of the edges that it traverses:

$$L(P) = \sum_{i=1}^l w(e_i).$$

Here, the edge weights are thus interpreted as a distance between the vertices they connect. This definition allows to recover the one of walk length for unweighted graphs ([Definition 6.12](#)) by taking unit weights.

Definition 6.19 (Shortest path problem)

Given a simple, undirected and weighted graph $G = (V, E, \omega)$ with positive weights, and $s, t \in V$, a shortest path problem consists in finding the shortest path that connects s and t , that is:

$$\operatorname{argmin}_{P \in \mathcal{P}(s, t)} L(P),$$

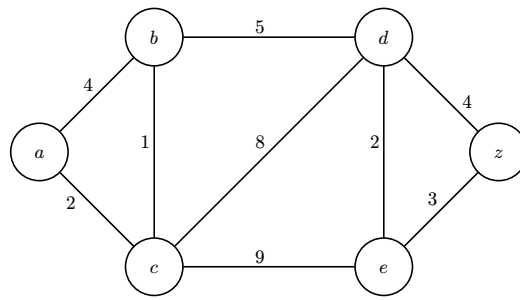
where $\mathcal{P}(s, t)$ is the set of paths that connect s and t .

? Question

In an unweighted graph, how can you get the shortest path length using the adjacency matrix?

If the path is long, this gets very costly, though. Also, if the graph is weighted, the shortest path in terms of number of traversed edges is not necessarily the shortest path in terms of edge weights. That’s why we need another method to compute shortest paths, such as Dijkstra’s algorithm. The basic idea of the algorithm is to iteratively explore the graph from the point of view of the starting vertex s . Starting from s , in each step we find its next closest vertex, until we’ve reached t . Crucially though, since shortcuts can sometimes appear, we need to keep track of known or estimated distances throughout the exploration. Let’s proceed on the following example.

Example. Find the shortest path from a to z in:



Algorithm 6.1: Dijkstra's

Input: $G = (V, E, \omega)$ connected and simple with positive weights, and with vertices $a = v_0, v_1, \dots, v_n = z$.

Output: $d(a, z)$, length of shortest path from a to z .

```

1 for  $i = 1$  to  $n$ 
2    $d(a, v_i) = \infty$                                 ▷ initialise unknown shortest lengths as infinity
3  $d(a, a) = 0$ 
4  $S = \emptyset$ 
5 while  $z \notin S$                                        ▷ until we've reached  $z$ 
6    $u =$  a vertex not in  $S$  with  $d(a, u)$  minimal        ▷ consider the next closest vertex
7    $S = S \cup \{u\}$ 
8   for all  $v \in \mathcal{N}(u)$                                ▷ for all neighbours  $v$  of  $u$ 
9     if  $d(a, u) + w(u, v) < d(a, v)$                    ▷ if going through  $u$  creates a shortcut to reach  $v$ 
10    then  $d(a, v) = d(a, u) + w(u, v)$                  ▷ update the shortest length to reach  $v$ 
```

Remark

- If at Line 6 there are several options, we can choose any of them.
- The sequence of vertices composing the path can be obtained by keeping track of the current best path, or simply of the best predecessor, whenever we reassign $d(a, v)$ in Line 10.
- The shortest path between two vertices is not necessarily unique. It is perfectly possible to have two or more paths of equal length between a given pair of vertices.

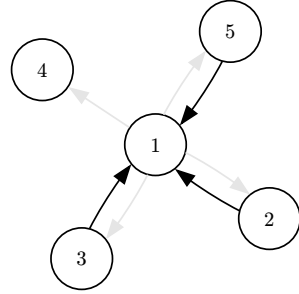
6.3 Describing graphs

6.3.1 Degree

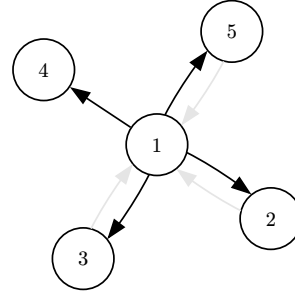
A vertex' importance can first be quantified by its degree.

Definition 6.20

Let G be a directed graph $G = (V, E)$, and let $v \in V$ be a vertex of G . The **in-degree** $k^{(\text{in})}(v)$ of v is the number of edges whose head is v , that is which end at v . The **out-degree** $k^{(\text{out})}(v)$ of v is the number of edges whose tail is v , that is which start from v .



Computing 1's in-degree



Computing 1's out-degree

Proposition 6.21

Let's consider a directed graph G and its adjacency matrix A associated to the ordering $v_1, v_2, \dots, v_{|V|}$ of its vertex set V . Then:

$$\forall i \in \llbracket 1, |V| \rrbracket, \begin{cases} k^{(\text{in})}(v_i) = \sum_j A_{ji} \\ k^{(\text{out})}(v_i) = \sum_j A_{ij} \end{cases}$$

It then follows that summing over degrees amounts to summing all entries of the adjacency matrix, which gives the following result.

Proposition 6.22

In any directed graph $G = (V, E)$:

$$\sum_{v \in V} k^{(\text{in})}(v) = \sum_{v \in V} k^{(\text{out})}(v) = |E|$$

The definition of degree follows naturally for undirected graphs, by considering that two directed edges (u, v) and (v, u) correspond to a single undirected edge $\{u, v\}$.

Definition 6.23

The **degree** of a vertex $v \in V$ in an undirected graph $G = (V, E)$ is the number of edges incident with it, except that a loop contributes twice to the degree of that vertex. The degree of a vertex v is denoted by $k(v)$.

Proposition 6.24

Let's consider an undirected graph G and its adjacency matrix A associated to the ordering $v_1, v_2, \dots, v_{|V|}$ of its vertex set V . Then:

$$\forall i \in \llbracket 1, |V| \rrbracket, k(v_i) = A_{ii} + \sum_j \frac{A_{ij} + A_{ji}}{2} = A_{ii} + \sum_j A_{ij}$$

We can then get a similar result as [Proposition 6.22](#), which is known as the handshaking theorem.

Theorem 6.25 (The handshaking theorem)

In any undirected graph $G = (V, E)$, we have that

$$\sum_{v \in V} k(v) = 2|E|$$

Remark

This also holds for undirected graphs with loops, thanks to the convention we took to consider a self-loop as incident twice to its vertex.

Corollary 6.25.1

For any undirected graph, the sum of all vertex degrees is an even number.

Corollary 6.25.2

Any undirected graph has an even number of vertices of odd degree.

Corollary 6.25.3

For any undirected graph with an odd number of vertices, there is an odd number of vertices of even degree.

The notion of degree also allows us to define a very special kind of graph.

Definition 6.26 (Regular graph)

A **regular graph** is an undirected graph whose vertices all have the same degree.

The notion of degree also gets naturally generalised to the case of weighted graphs.

Definition 6.27

Let G be a weighted directed graph $G = (V, E, \omega)$, and let $v \in V$ be a vertex of G . The **in-strength** $s^{(\text{in})}(v)$ of v is the sum of the weights of edges whose head is v . The **out-strength** $s^{(\text{out})}(v)$ of v is the sum of the weights of edges whose tail is v .

Definition 6.28

Let G be a weighted undirected graph $G = (V, E, \omega)$, and let $v \in V$ be a vertex of G . The **strength** $s(v)$ of v is the sum of the weights of edges adjacent to v , except that a loop contributes its weight twice.

[Proposition 6.21](#) then also holds for strengths in weighted directed graphs, and [Proposition 6.24](#) for those in weighted undirected ones.

These measures, degree and strength, are very local in nature, and thus may not represent the importance of a vertex in the context of the whole graph.

6.3.2 Eigenvector centrality

A first way to provide more global information to quantify a vertex' importance is to consider that a vertex should be considered more important not only the more neighbours it has, but also the more important these neighbours are. That is how we define the eigenvector centrality $x(u)$ of a vertex u in an undirected graph: it's proportional to the sum of the centralities of its neighbours:

$$x(u) = c^{-1} \sum_{v \in \mathcal{N}(u)} x(v),$$

where $c \in \mathbb{R}^+$ is a constant. Rewriting this equality for every vertex, and involving the adjacency matrix A of the graph, we get

$$x = c^{-1}Ax \Leftrightarrow Ax = cx,$$

where x is a vector of centrality scores, which is an eigenvector of A associated to the eigenvalue c ! And from the Perron-Frobenius theorem from linear algebra, we know that since A has non-negative values, if we want our centralities to all be non-negative, c must be the largest eigenvalue.

Definition 6.29 (Eigenvector centrality)

Given a simple undirected graph represented by an adjacency matrix A , the eigenvector centrality of vertex i is the i^{th} element of one of A 's leading eigenvectors.

Remark

- The point of centralities is to compare vertices, so the same eigenvector should be considered for all nodes!

- To generalise to directed networks, some small tweaks are needed: see Sections 7.1.2-3 in Newman.

6.3.3 Betweenness centrality

A second way to account for the global graph context to quantify a vertex' centrality is to use shortest paths.

Definition 6.30 (Vertex betweenness: simple)

The betweenness centrality of a vertex is the number of shortest paths between pairs of other vertices which pass through it.

However, as mentioned above, there may be more than one shortest path between a pair of vertices.

Definition 6.31 (Vertex betweenness: full)

The betweenness centrality of a vertex is the number of shortest paths between pairs of other vertices which pass through it, where we weight each path by the number of shortest paths between each pair of vertices:

$$\forall u \in V, b(u) = \sum_{s, t \in V \setminus \{u\}} \sum_{P \in \mathcal{P}(s, t)} \sum_{u \in P} \frac{1}{|\mathcal{P}(s, t)|},$$

where $\mathcal{P}(s, t)$ is the set of shortest paths from s to t .

A strength of betweenness is that it not only characterises the importance of vertices, but of edges too!

Definition 6.32 (Edge betweenness)

The betweenness centrality of an edge is the number of shortest paths passing through it, where we consider paths between all connected vertex pairs, weighting multiple shortest paths appropriately.

Remark

This edge centrality is linked to the “[strength of weak ties](#)” theory formulated by the sociologist [Mark Granovetter](#), which suggests that weak social relationships are central for the diffusion of information in a network (in very rough terms).

6.3.4 Graph size

The size of a graph can be simply considered to be its number of vertices and edges, but this gives little information. Indeed, it does not say how far away vertices are from each other in the graph. For that, we can look into the lengths of shortest paths, and for instance compute the average shortest path length. Alternatively, we can consider its longest shortest path.

Definition 6.33 (Graph diameter)

The diameter of a graph is the length of its longest shortest path, or in other words, the distance between its farthest vertices, in terms of shortest path length.

Remark

The graph diameter is what's behind the idea of the “six degrees of separation”: the fact that if you build the graph of social connections between people, its diameter is equal to 6 (or at least, in a famous experiment made by Stanley Milgram, for more see [this video](#)).

6.3.5 Graph density

We already saw how to check how well connected a graph is, using cut edges and vertices ([Definition 6.16](#)). A complementary, easier way to quantify that is to count how many edges the graph has, and compare it to the maximum it could have.

Question

How many edges can a simple undirected graph have at maximum?

Hint: this is a combinatorics question!

Definition 6.34 (Graph density)

The density ρ of a graph is the fraction of possible edges which are actually present in the graph.

Remark

The density can be thought of as the probability that two vertices picked from the graph uniformly at random are connected by an edge.

Proposition 6.35

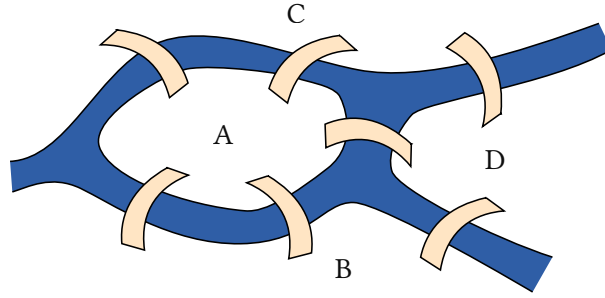
For simple undirected graphs,

$$\rho = \frac{2m}{n(n-1)}.$$

6.4 Euler and Hamilton paths

6.4.1 Eulerian graphs

Problem: The old city of Königsberg was crossed by a river and there were seven bridges. Was it possible to start walking at some point of the city and get back to the same place by crossing every bridge exactly once?



In graph theory terms, this gives: given a graph $G = (V, E)$, is there any circuit containing every edge $e \in E$? (If it is a circuit, then each edge is visited exactly once).

Definition 6.36

An **Euler tour** is a circuit containing every edge of the graph. A graph admitting an Euler tour is an **Eulerian graph**.

An **Euler trail** is an open trail that contains all the edges of the graph.

Theorem 6.37

A connected graph is Eulerian if and only if the degree of all its vertices is even. A connected graph contains an Euler trail if and only if it contains exactly two vertices of odd degree. A connected and directed graph is Eulerian if and only if for every vertex $v \in V$, $d_i(v) = d_o(v)$.

Therefore, the **problem of the bridges of Königsberg** does not have any solution: the corresponding graph does not admit any Euler tour/trail.

6.4.2 Fleury's algorithm

Let $G = (V, E)$ be a connected graph with all its vertices of even degree:

- (i) **Initial step:** We choose any vertex v_0 as the initial vertex of the Euler tour $C_0 = (v_0)$ and we define $G_0 = (V_0, E_0) = G$. The algorithm sequentially increases the tour C_0 , while it sequentially deletes elements from G .
- (ii) **How to extend the trail?:** Let $C_i = (v_0, e_1, v_1, \dots, e_i, v_i)$ be the trail corresponding to the graph $G_i = (V_i, E_i) \subseteq G_0$.
 - If there exists a unique edge incident with v_i , $e_{i+1} = \{v_i, w\} \in E_i = E \setminus \{e_1, e_2, \dots, e_i\}$:
 - $C_{i+1} = (v_0, e_1, v_1, \dots, e_i, v_i, e_{i+1}, w)$.
 - $G_{i+1} = (V_i \setminus \{v_i\}, E_i \setminus \{e_{i+1}\}) = (V_{i+1}, E_{i+1})$.
 - If there are several edges in E_i incident with v_i , we can choose any of these edges as long as the chosen one is not a bridge of G_i . If we choose $e_{i+1} = \{v_i, w\} \in E_i$:
 - $C_{i+1} = (v_0, e_1, v_1, \dots, e_i, v_i, e_{i+1}, w)$.
 - $G_{i+1} = (V_i, E_i \setminus \{e_{i+1}\}) = (V_{i+1}, E_{i+1})$.
- (iii) We repeat Step (2) $|E|$ times until $G_{|E|} = (\emptyset, \emptyset)$. Then $C_{|E|}$ is the Euler tour we were looking for.

6.4.3 Hamiltonian graphs

Problem 6: Is it possible to find a cycle on a graph G such that it contains all vertices of G exactly once?

Definition 6.38

A **Hamilton cycle** of a graph G is a cycle that contains all the vertices of G . A graph admitting one Hamilton cycle is a **Hamiltonian graph**.

A **Hamilton path** of a graph G is an open path that contains all vertices of G .

The problem of deciding that a given graph is Hamiltonian or not is hard.

Theorem 6.39 (Dirac, 1950)

If G is a simple graph with $n \geq 3$ vertices and each vertex has a degree $\geq n/2$, then G is a Hamiltonian graph.

Remark. Not every Hamiltonian graph satisfies the above condition: e.g. C_n with $n \geq 5$.

6.5 Trees

6.5.1 Definitions

Definition 6.40

A **tree** is a simple connected graph with no cycles. A **forest** is a simple graph with no cycles. Each connected component of a forest is a tree.

Remark. Trees may be rooted trees. A rooted tree is a tree with one distinguished vertex (the root). Hereafter, we will assume that all trees are rootless, unless specified.

Theorem 6.41

- (i) The simple graph G is a tree if and only if it is connected and, if we remove any edge, we obtain a disconnected graph.
- (ii) The simple graph G is a tree if and only if it does not contain any cycles and, if we add any edge, we create a cycle.

Theorem 6.42

A graph $G = (V, E)$ is a tree if and only if there exists a unique path between any pair of vertices.

Definition 6.43

How to grow a tree?

- (i) Start from the trivial tree $T = (\{r\}, \emptyset)$, where r is the root vertex.

(ii) Given $T = (V, E)$, add a new vertex u and a new edge $\{u, v\}$ where $v \in V$.

Theorem 6.44

Any graph obtained by using the preceding procedure is a tree, and any tree can be obtained in this way.

6.5.2 Properties

Theorem 6.45

Any tree with at least two vertices contains at least two vertices of degree one.

Theorem 6.46

Any tree with n vertices has $n - 1$ edges.

Theorem 6.47

If G is a graph with n vertices, then the following statements are equivalent:

- (i) G is a tree.
- (ii) G is connected and has $n - 1$ edges.
- (iii) G has $n - 1$ edges and does not contain any cycle.

6.5.3 Minimum-weight spanning tree

Definition 6.48

A **spanning tree** of a connected graph G is a subgraph of G that is a tree and contains all vertices of G .

Definition 6.49

A **minimum-weight spanning tree** of a connected weighted graph $G = (V, E, \omega)$ is a spanning tree $T = (V, A)$ of G such that $\omega(A) = \sum_{e \in A} \omega(e)$ takes the minimum possible value.

Problem 1: Find a minimum-weight spanning tree of a connected weighted graph $G = (V, E, \omega)$.

Remark. The number of trees on n vertices grows very rapidly with n .

Definition 6.50

A **greedy algorithm** to solve a given problem is an algorithm such that at every step, it always takes, among all the choices allowed by the problem, the optimum one.

Algorithm 6.2: Prim's algorithm

```
1  $T_1 = (V_1, E_1)$  where  $E_1 = \{e_1\}$ ,  $e_1 = \{x_0, x_1\}$  is one edge with minimum weight  $\omega_{\min}$ ,  
   and  $V_1 = \{x_0, x_1\}$ .  
2 for  $i = 1$  to  $n - 2$   
3    $e_{i+1} = \{x_i, x_{i+1}\}$  edge of minimum weight that is incident with a vertex  $x_j$  of  $T_i =$   
    $(V_i, E_i)$ , and such that it does not form a cycle when added to  $T_i$   
4    $T_{i+1} = (V_i \cup \{x_{i+1}\}, E_i \cup \{e_{i+1}\}) = (V_{i+1}, E_{i+1})$ 
```

Remarks.

- The edge e_i ($i = 1, \dots, n - 1$) might not be unique.
- The minimum-weight spanning tree might not be unique.
- At each step, T_i is a tree ($1 \leq i \leq n - 1$).

Theorem 6.51

Given a connected weighted graph $G = (V, E, \omega)$, Prim's algorithm produces a minimum-weight spanning tree of G .

Algorithm 6.3: Kruskal's algorithm

```
1  $T_0 = (V, E_0)$  with  $E_0 = \emptyset$   
2 for  $i = 1$  to  $n - 1$   
3    $e_i =$  edge of minimum weight such that it does not form a cycle when added to  $T_{i-1} =$   
    $(V, E_{i-1})$   
4    $T_i = (V, E_{i-1} \cup \{e_i\}) = (V, E_i)$ 
```

Remarks.

- The edge e_i ($i = 1, \dots, n - 1$) might not be unique.
- At each step, T_i is a forest ($1 \leq i \leq n - 1$).

Theorem 6.52

Given a connected weighted graph $G = (V, E, \omega)$, Kruskal's algorithm produces a minimum-weight spanning tree of G .